



Plataforma web de jogos de geolocalização para treino em OSINT

GeoExplorer

RELATÓRIO FINAL

Marcos Monteiro · 1902045
Orientador · Pedro Pestana

Junho de 2026

Índice

<i>Lista de Figuras</i>	3
<i>Lista de Tabelas</i>	4
<i>Resumo</i>	5
1. Introdução	6
1.1. Enquadramento	6
1.2. Necessidades identificadas	7
1.3. Objetivos Finais	8
1.4. Âmbito e Limitações	9
1.5. Organização do relatório	9
2. Desenho e Arquitetura	11
2.1. Tecnologias usadas	11
2.2. Arquitetura do sistema	12
2.3. Modelo de dados	13
2.4. Fontes visuais e dataset	14
2.5. Fluxos e lógica de jogo	15
2.6. Decisões de arquitetura	15
2.7. Privacidade, segurança e dados	16
3. Implementação	18
3.1. Evolução desde o relatório intercalar	18
3.2. Frontend e experiência de jogo	19
3.3 Backend e regras de jogo	25
3.4 Database, schema e persistência	25
3.5 Expansão e auditoria do dataset	26
3.6 Multiplayer	28
3.7 Docker, logs e versão pública	29
3.8 Apoio de IA	31
4. Testes e Validação	32
4.1 Estratégia de validação	32
4.2 Validação automática	33
4.3 Validação manual	34
4.4 Auditoria do dataset	35

4.5 Riscos e segurança	35
4.6 Verificação do relatório	36
5. Conclusões	37
5.1 Resultado final	37
5.2 Limitações	38
5.3 Trabalho futuro	39
5.4 Reflexão pessoal.....	40
Bibliografia.....	42
Anexo A - Excertos Técnicos Seleccionados.....	43
A1. Registo da API, CORS, database e SignalR	43
A2. Comunicação entre frontend e API no modo real	43
A3. Validação de palpite no backend.....	44
A4. Catálogo PostgreSQL antes do seed JSON	44
A5. Schema EF das tabelas principais	45
A6. Eventos em tempo real do hub multiplayer	45
A7. Perfil full do Docker Compose	45
Anexo B - Comandos de validação	46

Lista de Figuras

Figura 1 - Diagrama C4 de contexto do GeoExplorer.	12
Figura 2 - Diagrama C4 de contentores.	12
Figura 3 - Modelo de dados relacional final em PostgreSQL.	14
Figura 4 - Página inicial e identidade visual final.	20
Figura 5 - Configuração de uma missão solo com país, tempo e pontuação total.	21
Figura 6 - Ronda com imagem real, briefing e pontuação acumulada.	22
Figura 7 - Mapa real aberto para marcar a posição.	22
Figura 8 - Palpite marcado antes da submissão.	23
Figura 9 - Debrief da ronda com distância, pontuação da ronda e total acumulado.	23
Figura 10 - Resultado final da sessão solo com resumo e mapa da missão.	24
Figura 11 - Entrada multiplayer com criação ou entrada em sala.	28
Figura 12 - Lobby multiplayer com jogadores, país, tempo e pontuação total.	29

Lista de Tabelas

Tabela 1 - Objetivos finais e evidência de cumprimento	8
Tabela 2 - Tecnologias utilizadas e justificação.....	11
Tabela 3 - Entidades principais do modelo de dados.....	13
Tabela 4 - Fontes visuais do projeto.....	14
Tabela 5 - Decisões de arquitetura existentes.....	16
Tabela 6 - Dificuldades, feedback e resposta técnica.....	18
Tabela 7 - Ecrãs principais e estado final.....	20
Tabela 8 - Uso de IA e responsabilidade final.....	31
Tabela 9 - Validação automática realizada.	33
Tabela 10 - Validação manual e operacional.	34
Tabela 11 - Riscos finais e mitigação.	35
Tabela 12 - Limitações finais.	38
Tabela 13 - Excertos técnicos selecionados.....	43

Resumo

O GeoExplorer é uma aplicação web de geolocalização para treino de observação visual, inferência geográfica e leitura de contexto em cenários próximos de OSINT. O jogador observa uma imagem real, interpreta pistas visuais, marca uma posição num mapa europeu e recebe feedback através da distância ao local correto e da pontuação obtida.

Na versão final realizei uma evolução significativa face ao relatório intercalar. O projeto passou de uma base jogável com 158 locais reais para uma aplicação com frontend React(TS), backend em .NET, PostgreSQL em Docker, migrations com Entity Framework Core, 10 975 locais reais e uma extensão multiplayer com SignalR.

A decisão técnica mais importante da fase final foi assumir API + PostgreSQL como fluxo real. No início usei o modo mock para avançar depressa no frontend, mas tive problemas previsíveis quando o dataset cresceu, porque carregar um ficheiro completo de locais no browser deixaria a aplicação pesada e menos sustentável. Por isso passei a seleção de rondas para o backend e deixei o mock apenas como apoio de desenvolvimento.

Também trabalhei na parte visual e no tratamento dos dados. O dataset final combina 5 513 locais com imagens Wikimedia Commons e 5462 panoramas Panoramax como fontes principais jogáveis, mantendo licenças e atribuições. Mantive Mapillary como fonte opcional resolvida pelo backend quando há token local, para não expor credenciais nem tornar a demonstração dependente de uma API externa.

A validação juntou testes automáticos, build frontend, auditoria do dataset, validação Docker, revisão de dependências, testes manuais na versão pública e testes com utilizadores externos. O resultado é uma aplicação jogável, com funcionamento técnico claro e limitações assumidas, como não ter contas de utilizador, manter o âmbito europeu, usar Mapillary como fonte opcional, ter um multiplayer adequado à demonstração académica e manter o deployment público ainda experimental.

1. Introdução

1.1. ENQUADRAMENTO

O GeoExplorer nasceu da ideia de construir uma aplicação web interativa para treinar geolocalização a partir de imagens. Este tipo de treino exige observação, leitura de pistas visuais, comparação de padrões urbanos, atenção à vegetação, arquitetura, relevo e sinais culturais. Estas competências aparecem frequentemente associadas a OSINT, investigação digital, jornalismo de verificação e análise de informação aberta.

Desde o início evitei apresentar o projeto como uma alternativa completa a plataformas comerciais. O objetivo foi mais realista e adequado ao contexto académico. Quis construir um MVP jogável, com locais reais na Europa, mapa interativo, cálculo de distância e pontuação, registo de resultados e documentação técnica suficiente para explicar as decisões. A fase final acrescentou mais ambição, mas sem abandonar este núcleo.

O relatório intercalar já servia como base do relatório final. Na altura, tinha um frontend jogável, backend inicial, PostgreSQL em preparação e um conjunto pequeno de locais reais. No final, o projeto ficou mais completo e verificável. A database passou a ser criada por migrations, o catálogo foi aumentado para 10 975 locais, o modo API passou a ser o fluxo principal da aplicação, o Docker foi validado e o multiplayer foi implementado como extensão controlada.

A direção visual também evoluiu. Usei uma linguagem inspirada em interfaces táticas e menus de missão de séries de jogos como Splinter Cell, Call of Duty, 007 e Metal Gear Solid, com verde forte, contraste alto, noção de missão, briefing e debrief. Esta escolha não foi apenas estética. Também ajudou a ligar o tema OSINT/geolocalização a uma experiência focada em leitura rápida de informação, mapa e decisão.

O projeto não pretende competir com plataformas como o GeoGuessr. A diferença principal está no contexto e no objetivo. O GeoExplorer foi desenvolvido como trabalho académico, com código próprio, decisões técnicas documentadas, dados auditáveis e uma demonstração controlada. Em vez de tentar reproduzir uma plataforma comercial completa, o foco passou por construir uma versão explicável, executável e coerente com o tempo da unidade curricular.

Esta distinção é importante para a defesa do projeto. Plataformas comerciais resolvem problemas de escala, produto, pagamentos, moderação, contas, rankings globais e disponibilidade internacional. O GeoExplorer concentrou-se no que era possível demonstrar tecnicamente. O valor está na ligação entre frontend, backend, database, dataset real, testes, documentação e decisões de arquitetura que podem ser justificadas perante um júri.

1.2. NECESSIDADES IDENTIFICADAS

A principal necessidade do projeto é criar um ambiente simples para praticar inferência geográfica com feedback rápido. Uma pessoa pode tentar adivinhar locais a partir de imagens, mas sem distância, pontuação e comparação com o local correto fica difícil perceber se a hipótese estava próxima ou afastada.

- Treinar observação visual aplicada a imagens reais.
- Dar feedback quantitativo através de distância e pontuação.
- Permitir sessões curtas com várias rondas.
- Manter o âmbito europeu para reduzir risco de dados e licenças.
- Documentar fontes visuais, licenças e atribuições usadas no jogo.
- Documentar decisões técnicas de forma defensável para avaliação académica.

1.3. OBJETIVOS FINAIS

O objetivo geral do GeoExplorer foi desenvolver uma aplicação funcional para sessões de geolocalização na Europa. No estado final, o jogador consegue iniciar uma sessão, jogar várias rondas, observar imagens reais, marcar palpites no mapa, receber distância e pontuação, consultar resultados e usar uma extensão multiplayer por salas.

Para tornar este objetivo verificável, mantive os critérios de aceitação definidos na proposta e associei cada um deles a evidência no código, nos testes e na aplicação final. A tabela seguinte mostra o que foi realizado e onde isso aparece no projeto.

Objetivo	Estado final	Evidência
Iniciar sessão individual	Implementado	Página inicial, setup e endpoint POST /api/sessions.
Configurar rondas e temporizador	Implementado	Modo livre ou cronometrado com validação no backend.
Mostrar locais europeus reais	Implementado	Dataset com 10 975 locais reais, coordenadas, fonte, licença e atribuição.
Submeter palpite no mapa	Implementado	Leaflet/OpenStreetMap no frontend e validação no backend.
Calcular distância e pontuação	Implementado	Distância Haversine e pontuação calculadas no backend.
Guardar sessões e rondas	Implementado	PostgreSQL com Entity Framework Core e migrations.
Mostrar resultado final	Implementado	Resumo final de sessão com pontuação acumulada.
Executar em Docker	Implementado	Perfil full com frontend, backend e database.
Multiplayer simples	Extensão implementada	SignalR, salas, dono da sala, rondas sincronizadas e persistência em PostgreSQL.

Tabela 1 - Objetivos finais e evidência de cumprimento

O núcleo definido na proposta foi cumprido. O multiplayer aparece como extensão porque não era obrigatório no MVP inicial, mas acabou por ser possível depois de o modo solo, a API e a persistência estarem estáveis.

A tabela de objetivos mostra o cumprimento funcional, mas o resultado final também deve ser lido em termos de consolidação técnica. O jogo não ficou apenas com ecrãs navegáveis. Ficou com validação no backend, persistência opcional em PostgreSQL, migrations, integração Docker, auditoria do dataset e separação entre modo mock e modo API. Esta evolução tornou o projeto mais fácil de explicar e mais próximo de uma aplicação real.

O objetivo de usar locais europeus reais ganhou mais peso na fase final. O dataset deixou de ser apenas um conjunto pequeno de exemplos e passou a ser uma parte central do projeto. A existência de 10 975 locais permitiu demonstrar variedade, filtro por país e maior capacidade de gerar rondas sem repetir sempre os mesmos sítios. Ao mesmo tempo, a auditoria ajudou a controlar coordenadas, fontes e licenças.

O multiplayer ficou fora do núcleo obrigatório, mas passou a funcionar como evidência adicional. A sua inclusão só fez sentido depois de o modo solo estar estável. Essa ordem foi importante, porque evitou construir uma funcionalidade social sobre uma base incompleta. No estado final, o multiplayer mostra que a arquitetura suporta interação em tempo real, mesmo sem assumir escala pública.

1.4. ÂMBITO E LIMITAÇÕES

Mantive a cobertura geográfica limitada à Europa. Esta escolha foi importante para controlar o projeto, porque permitiu melhorar a qualidade dos dados, reduzir incerteza nas fontes visuais e manter a demonstração mais previsível. A aplicação não precisa de login para demonstrar o fluxo principal, por isso não implementei autenticação nem histórico pessoal por utilizador.

No relatório intercalar, o âmbito estava definido como individual, europeu e sem autenticação. No final, mantive essas restrições no núcleo do projeto. Não implementei ranking público, contas de utilizador nem cobertura global. O multiplayer só entrou depois de o fluxo principal estar concluído e ficou como extensão controlada, não como dependência do MVP.

No multiplayer, optei por salas por link, salas públicas, password opcional e nomes únicos dentro da sala. Esta solução é suficiente e evitou introduzir uma camada de autenticação antes de haver uma necessidade real. Se o projeto continuasse, podia ser acrescentado contas de utilizador.

Também mantive Mapillary como fonte opcional. Tive de tratar este ponto com cuidado porque a API exige token e devolve URLs temporárias para miniaturas. Por isso guardei caminhos estáveis do backend, como `/api/media/mapillary/{id}`, e deixei o backend resolver a imagem quando há token local.

A escalabilidade foi tratada como limitação assumida. A aplicação demonstra o fluxo completo, mas não afirma estar pronta para uma utilização pública com muitos utilizadores em simultâneo. Para esse cenário seria necessário load testing, observabilidade mais completa, limites de utilização, infraestrutura gerida, estratégia de cache e adaptação do multiplayer para múltiplas instâncias.

1.5. ORGANIZAÇÃO DO RELATÓRIO

O Capítulo 2 descreve a arquitetura e as decisões técnicas. O Capítulo 3 explica a implementação e a evolução desde o intercalar até ao estado final. O Capítulo 4 reúne testes, validação e riscos. O Capítulo 5 apresenta conclusões, limitações e trabalho futuro. Nos anexos incluí excertos técnicos curtos, porque o código completo pertence ao repositório e não deve ser colado integralmente no relatório.

A estrutura foi pensada para que o relatório final não seja apenas uma atualização simples do intercalar. O intercalar mostrou o ponto de partida, com uma aplicação já jogável, mas ainda com várias hipóteses por validar. Este relatório mostra a fase final do projeto, incluindo o que ficou implementado, o que mudou por necessidade técnica, onde tive problemas, que validações fiz e que limitações continuam assumidas.

Também procurei manter um cunho pessoal no texto. Em vez de escrever como se o projeto tivesse aparecido pronto, explico onde comecei, que decisões tive de rever e

porque algumas ideias ficaram como trabalho futuro. Isto é importante porque o valor do projeto não está apenas no resultado, está também na evolução técnica e na capacidade de justificar escolhas.

A comparação com o intercalar é simples. Na fase intermédia ainda estava a provar que o jogo podia funcionar de ponta a ponta. Na fase final, tive de provar que o projeto estava preparado para avaliação, conseguindo correr em Docker, guardar dados numa database, lidar com um dataset grande, explicar fontes e licenças, validar multiplayer.

2. Desenho e Arquitetura

Este capítulo descreve o desenho técnico final do GeoExplorer. A arquitetura deixou de ser apenas uma proposta e passou a estar validada numa aplicação com frontend React, backend .NET, PostgreSQL, Docker, dataset real e extensão multiplayer por SignalR.

2.1. TECNOLOGIAS USADAS

Área	Tecnologia	Justificação
Frontend	React, TypeScript, Vite	Permitiu iterar rapidamente numa interface de jogo com estado, ecrãs e interfaces tipadas.
Mapa	Leaflet/OpenStreetMap	Usei um mapa real para recolher coordenadas sem criar um motor cartográfico próprio.
Backend	.NET 8	Centralizei regras de jogo, validações, endpoints HTTP e SignalR numa API própria.
Database	PostgreSQL	O modelo relacional encaixa bem em catálogo, sessões, rondas, salas e palpites.
Data access	Entity Framework Core	Usei migrations para manter o schema alinhado com o código C#.
Tempo real	SignalR	A lógica multiplayer precisava de estado de sala, timers, palpites e envio de resultados em tempo real.
Execução	Docker Compose	Tornou a execução local mais reprodutível para desenvolvimento e avaliação.

Tabela 2 - Tecnologias utilizadas e justificação.

A stack manteve-se próxima da proposta aprovada, mas ficou mais concreta durante o desenvolvimento. Comecei por validar o fluxo com frontend e mock, depois liguei backend, database e Docker. Esta ordem ajudou-me a não ficar bloqueado pela infraestrutura antes de ter uma experiência jogável.

2.2. ARQUITETURA DO SISTEMA

Esta secção apresenta a arquitetura do GeoExplorer através de uma visão C4 simplificada. No nível de contexto, o GeoExplorer é o sistema principal usado pelo jogador e pelo professor/avaliador. OpenStreetMap fornece tiles do mapa, Wikimedia Commons, Panoramax e Mapillary entram como fontes visuais ou fontes de preparação de dados. O jogo, no entanto, usa um dataset local importado para PostgreSQL, reduzindo dependência externa durante a demonstração.

GeoExplorer - C4 Contexto

Entrega 1 - visão de sistema, utilizador e dependências externas

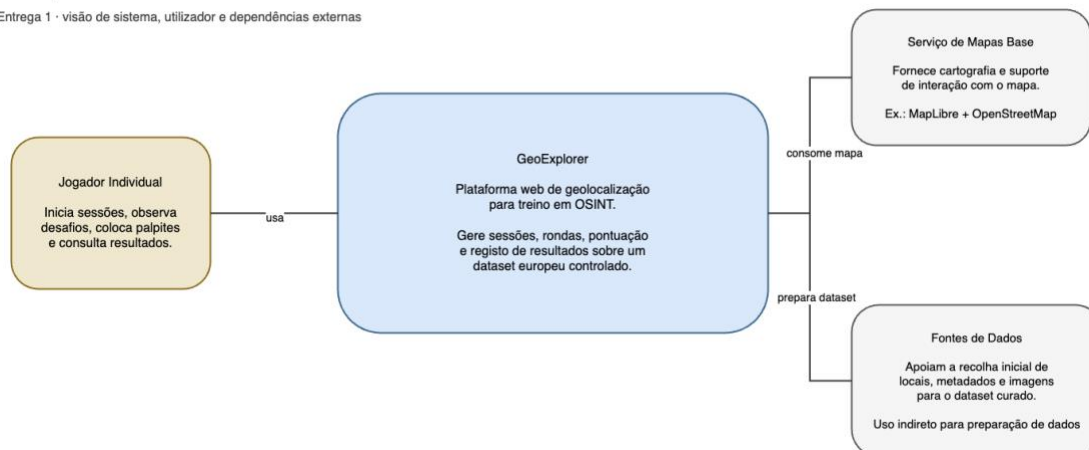


Figura 1 - Diagrama C4 de contexto do GeoExplorer.

No nível de contentores, o frontend comunica com a Minimal API por HTTP e com o Hub SignalR por WebSocket. O backend centraliza regras e persistência através de Entity Framework Core. O PostgreSQL guarda catálogo, sessões solo, rondas, salas multiplayer, jogadores e palpites.

GeoExplorer - C4 Contentores final

Frontend, backend, SignalR, Entity Framework, PostgreSQL e fontes externas

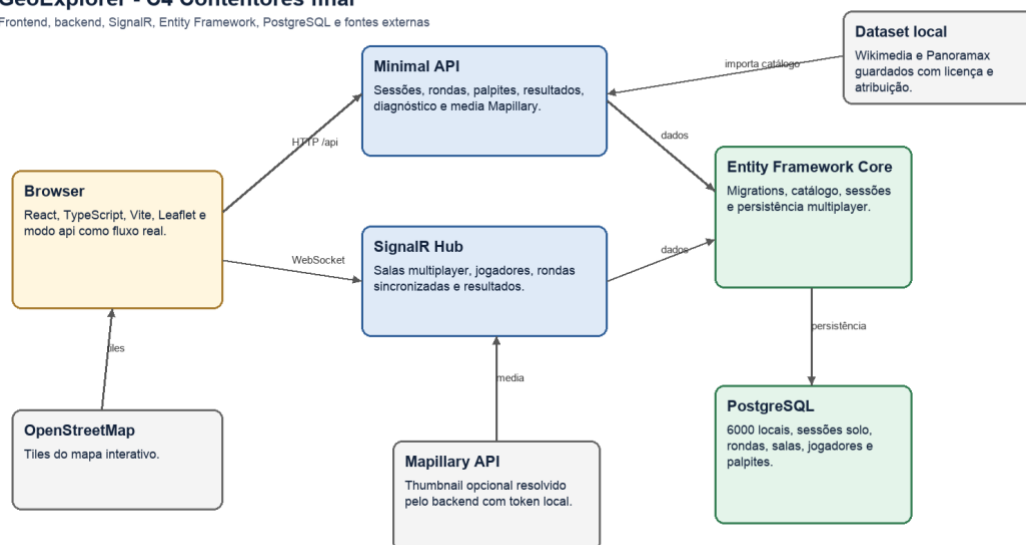


Figura 2 - Diagrama C4 de contentores.

O frontend não calcula a verdade do jogo quando está ligado à API. Pode apresentar estado, recolher interação e mostrar mapas, mas a validação de coordenadas, distância, pontuação e persistência fica do lado do backend. Esta escolha reduz inconsistências entre browser e servidor e facilita a defesa das regras de jogo, porque existe um ponto central para explicar a lógica.

A database foi mantida como parte do sistema e não apenas como armazenamento auxiliar. O schema representa sessões, rondas e palpites de forma coerente com o fluxo do jogo. Isto permitiu recuperar sessões guardadas, validar relações entre entidades e deixar o projeto preparado para histórico futuro, mesmo sem implementar contas de utilizador nesta entrega.

2.3. MODELO DE DADOS

Uma das mudanças importantes face ao início foi deixar o ficheiro SQL como apoio documental e passar a criar o schema por migrations do Entity Framework. Isto aproximou a database do código C# e reduziu o risco de o modelo documentado ficar diferente do modelo real.

Tabela	Finalidade	Notas
locations	Catálogo de locais reais.	Inclui coordenadas, media, licenças, pistas e visualSources em jsonb.
game_sessions	Sessões solo.	Guarda configuração, pontuação e estado.
session_rounds	Rondas solo.	Guarda local escolhido, visual source, palpite, distância e pontuação.
multiplayer_rooms	Salas multiplayer.	Inclui room code, dono da sala, estado, visibilidade e password_hash.
multiplayer_players	Jogadores da sala.	Guarda display name, ownership, ligação e pontuação total.
multiplayer_rounds	Rondas multiplayer.	Guarda a ronda sincronizada e a fonte visual escolhida.
multiplayer_guesses	Palpites multiplayer.	Um palpite por jogador/ronda, com distância e pontuação.

Tabela 3 - Entidades principais do modelo de dados.

GeoExplorer - Modelo de dados final

Tabelas principais criadas por migrations do Entity Framework

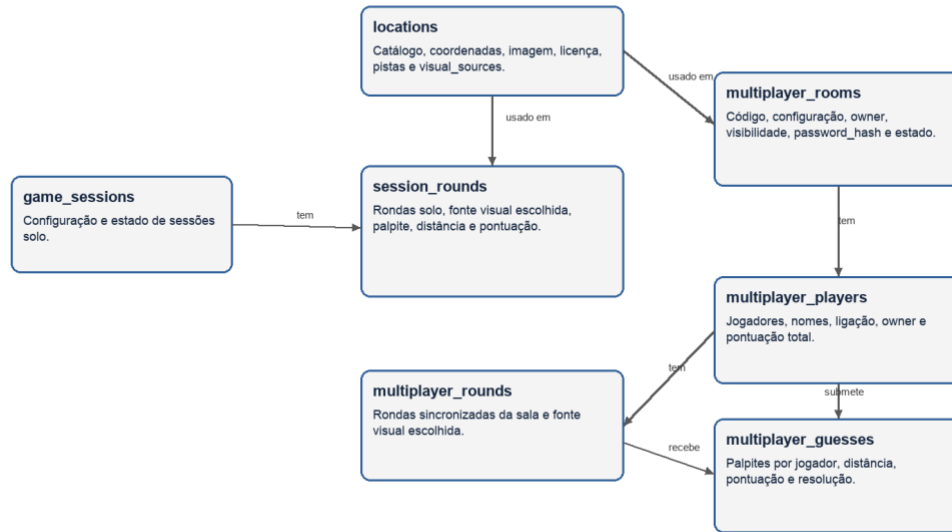


Figura 3 - Modelo de dados relacional final em PostgreSQL.

O campo visual_source nas rondas é especialmente importante. Quando o backend escolhe a imagem de uma ronda, guarda essa escolha. Assim, se a sessão for recuperada a partir da database, o resultado continua coerente com a fonte visual originalmente apresentada ao jogador.

2.4. FONTES VISUAIS E DATASET

No intercalar, o dataset ainda era pequeno. Na fase final, uma parte relevante do trabalho foi crescer o catálogo sem perder controlo de qualidade. Tive de equilibrar quantidade, licenciamento, diversidade visual e dificuldade do jogo. O resultado final tem 10 975 locais reais.

Fonte	Uso atual	Risco tratado
Wikimedia Commons	5 513 imagens principais.	Licença, atribuição e página de origem guardadas no dataset.
Panoramax	5 462 panoramas 360 como imagem principal jogável.	Fonte aberta útil para contexto de rua, mas com revisão visual.
Mapillary	1844 fontes opcionais.	Token fica no backend, URLs temporárias não entram no dataset.
OpenStreetMap	Mapa interativo.	Dependência externa para tiles, aceitável para demonstração controlada.

Tabela 4 - Fontes visuais do projeto.

O ponto técnico mais delicado foi evitar que o jogo dependesse de APIs externas. A recolha e validação podem usar serviços externos, mas a execução principal deve conseguir criar rondas a partir do catálogo importado para PostgreSQL. Esta separação tornou o projeto mais demonstrável e mais fácil de explicar.

A expansão para 10 975 locais obrigou a tratar os dados como parte central do projeto. Não bastava somar entradas ao ficheiro. Foi necessário manter coordenadas válidas, país associado, fonte visual, licença, atribuição e uma forma de auditar problemas. Esta

preocupação ficou especialmente importante porque o jogo usa imagens reais e, por isso, a origem dos dados faz parte da qualidade da solução.

Panoramax tornou-se uma fonte principal porque oferece imagens de rua abertas e adequadas ao contexto europeu. Wikimedia Commons também foi mantida para locais e imagens com valor visual reconhecível. Mapillary ficou como fonte opcional, resolvida pelo backend, porque exige token e URLs temporárias. Esta divisão evita expor credenciais no frontend e reduz dependência externa durante a demonstração.

O filtro por país acrescentou uma utilidade prática ao dataset. Em vez de escolher sempre aleatoriamente em toda a Europa, o jogador pode restringir a sessão a um país. Esta funcionalidade também mostrou a importância de ter cobertura mínima por país. Com poucos locais, o filtro seria tecnicamente possível, mas a experiência ficaria repetitiva e menos convincente.

Portugal recebeu atenção especial porque fazia sentido reforçar locais nacionais num projeto desenvolvido neste contexto. A cobertura portuguesa ajuda a testar o jogo com utilizadores próximos, facilita a identificação de erros visuais e cria exemplos mais fáceis de discutir durante a apresentação. Ao mesmo tempo, a cobertura europeia foi mantida para não transformar o projeto num jogo apenas nacional.

2.5. FLUXOS E LÓGICA DE JOGO

O fluxo solo é o núcleo do MVP. O jogador cria uma sessão, recebe uma ronda, observa a imagem, marca um ponto no mapa, submete o palpite, recebe o resultado e consulta o relatório final. O backend valida coordenadas, calcula distância e pontuação, e guarda o estado quando PostgreSQL está ativo.

O fluxo multiplayer usa a mesma lógica base de ronda, mas acrescenta sala, dono da sala, jogadores, estado de ligação, timer e envio de eventos em tempo real. Usei SignalR porque o frontend precisa de receber atualizações de sala

2.6. DECISÕES DE ARQUITETURA

ADR / decisão	Alternativa considerada	Razão da escolha
ADR-001 - Frontend React + TypeScript	Angular ou JS sem framework	Menos overhead inicial e interfaces mais seguras entre mock, API e UI.
ADR-002 - ASP.NET Core e PostgreSQL	FastAPI, ficheiros locais ou backend-as-a-service	Coerência com a proposta, controlo da lógica do jogo e modelo relacional claro.
ADR-002 - PostgreSQL em Docker	Supabase completo ou Turso/libSQL	Ambiente relacional simples, local e demonstrável, uma solução hosted fica para futuro.
ADR-002 - SignalR no backend	Supabase Realtime	O multiplayer não é só atualização de tabelas, envolve salas, timers, palpites e regras de jogo.
ADR-003 - API + PostgreSQL como fluxo real	Catálogo completo no browser	Evitei empacotar milhares de locais no frontend.

ADR / decisão	Alternativa considerada	Razão da escolha
ADR-004 - Fontes visuais reais	Mapillary obrigatório na execução	Mantive dataset local e Mapillary opcional para não expor token nem depender de uma API externa.

Tabela 5 - Decisões de arquitetura existentes.

Estas decisões estão registadas nos ADRs do projeto, mas a tabela resume o essencial para que o relatório seja compreensível mesmo sem abrir esses ficheiros. A decisão PostgreSQL/Supabase/Turso foi especialmente importante. Não rejeitei Supabase ou Turso para sempre, mas decidi que nesta fase o melhor era PostgreSQL local em Docker, com eventual revisão futura baseada em métricas reais de uso.

A escolha de PostgreSQL foi também uma escolha de simplicidade operacional para a entrega. Supabase teria vantagens se precisasse já de autenticação gerida, Storage, Realtime simples ou Row Level Security. Turso/libSQL poderia ser interessante se o padrão real fosse muita leitura distribuída com necessidade de baixa latência geográfica. No entanto, para uma aplicação com backend próprio, migrations EF e modelo relacional claro, PostgreSQL foi a decisão mais direta.

Outra decisão importante foi não levar o multiplayer para fora do backend. Poderia ter tentado sincronizar tudo por tabelas ou por polling, mas isso espalharia regras entre frontend e database. Com SignalR, a sala, os jogadores, o dono da sala, o timer, os palpites e os resultados continuam coordenados pelo backend, que é onde já estão as regras do jogo.

A decisão de manter Mapillary no backend também foi importante. O token não deve estar no frontend, e as URLs de imagem não são tão estáveis como ficheiros locais ou fontes abertas diretamente registadas. Ao expor um endpoint próprio, o backend protege a credencial e mantém a interface independente dos detalhes da API externa.

As decisões de arquitetura também mostram onde o projeto poderia crescer. SignalR resolve bem a comunicação em tempo real numa instância. Para escala horizontal, seria preciso avaliar Redis backplane ou outra estratégia de distribuição. PostgreSQL local em Docker é suficiente para demonstração e desenvolvimento. Para operação contínua, uma hosted database e backups passariam a ser necessários.

2.7. PRIVACIDADE, SEGURANÇA E DADOS

O projeto não usa cookies próprios, analytics comercial ou recolha genérica de cliques e movimentos. A aplicação guarda no localStorage apenas dados pequenos do próprio jogo, como o tutorial concluído, a opção de mostrar a pontuação total durante a ronda, a retoma de sessão solo, a identificação local do jogador multiplayer e a retoma de sala. Os logs técnicos ficam no backend e servem para diagnóstico.

Na segurança prática, foquei-me em quatro pontos principais. Mantive o .env fora do Git, não expus o token Mapillary no frontend, configurei CORS com origens explícitas e validei no servidor os palpites, códigos de sala e regras multiplayer. Não transformei

isto num sistema de produção completo, mas deixei as decisões importantes documentadas e explicáveis.

Também separei métricas técnicas de analytics comportamental. O endpoint `/api/diagnostics/database` ajuda a perceber leituras e escritas da database, mas não serve para seguir cliques ou movimentos do utilizador. Esta distinção foi deliberada, porque numa entrega académica consigo demonstrar funcionamento técnico sem criar uma camada de tracking desnecessária.

Se a aplicação fosse publicada de forma duradoura, teria de rever política de privacidade, retenção de logs, consentimento para métricas e eventual autenticação. Para a versão final da UC, mantive a recolha proporcional ao funcionamento do jogo e documentei o que fica guardado.

3. Implementação

Este capítulo descreve o trabalho realizado desde o relatório intercalar até à entrega final. O foco deixou de ser apenas completar ecrãs e passou a ser fechar integração, persistência, dataset, Docker, multiplayer, versão pública e validação.

3.1. EVOLUÇÃO DESDE O RELATÓRIO INTERCALAR

Depois do relatório intercalar, o trabalho deixou de ser apenas completar ecrãs. Passei a lidar com problemas concretos de integração, dados, performance e demonstração. A evolução principal ficou organizada em várias frentes de trabalho, resumidas de seguida.

1. Fechei o fluxo solo com setup, ronda, mapa real, submissão, debrief e o resultado final.
2. Liguei o backend .NET ao PostgreSQL com Entity Framework Core e migrations.
3. Expandi o dataset para 10 975 locais reais com fonte, licença e atribuição.
4. Tratei Panoramax como fonte principal jogável e Mapillary como fonte opcional resolvida pelo backend.
5. Validei Docker em perfil full, com frontend em modo API, backend e PostgreSQL.
6. Implementei multiplayer com SignalR depois de o núcleo estar estável.
7. Corrigi problemas de interface encontrados em testes reais e na versão pública.

Dificuldade ou feedback	Impacto	Resposta
Crescimento do dataset	O frontend ficaria pesado se recebesse tudo.	Passei a seleção de rondas para o backend e PostgreSQL.
Fontes visuais heterogéneas	Imagens podiam não ter licença ou qualidade suficiente.	Criei ferramentas de auditoria e mantive revisão manual.
Feedback exterior sobre clareza	Alguns fluxos que para mim eram óbvios não eram claros para utilizadores externos.	Ajustei navegação, mensagens, estados de erro, botões e fluxo multiplayer.
Testes fora do ambiente local	Na versão pública apareceram problemas menos visíveis no meu computador.	Usei logs, health check, diagnóstico da database e testes com utilizadores externos, incluindo pessoas próximas.
Ideia inicial demasiado centrada no solo	O projeto podia ficar tecnicamente correto, mas pouco diferenciado.	Mantive o solo como núcleo e acrescentei multiplayer como extensão controlada.
Mapillary com URLs temporárias	Links podiam falhar mais tarde.	Guardei caminhos estáveis do backend e resolvi miniaturas na execução.
Multiplayer com estados concorrentes	Havia risco de inconsistência entre jogadores.	Usei SignalR, estado central no backend e regras de resolução.
Volumes PostgreSQL antigos	Migrations podiam colidir com tabelas antigas.	Documentei recriação de volume em desenvolvimento.

Tabela 6 - Dificuldades, feedback e resposta técnica.

Também considero importante apresentar os contratempos, porque eles explicam melhor a evolução do projeto do que uma lista simples de funcionalidades. Algumas decisões mudaram por necessidade técnica, como passar o catálogo para a database, e outras mudaram por feedback exterior, como melhorar mensagens, navegação, estados vazios e clareza do multiplayer.

A ideia inicial era mais próxima de um jogo individual de geolocalização. Com o avanço do projeto e com testes feitos fora do meu ambiente local, percebi que a aplicação ficava mais interessante se mantivesse o modo solo como base sólida, mas mostrasse também uma camada social curta. Foi por isso que o multiplayer entrou como extensão final e não como substituto do MVP.

A evolução semanal também ajudou a manter o projeto controlado. Nas primeiras semanas, o foco foi proposta, wireframes e arquitetura. Depois passei para frontend jogável e backend mínimo. A seguir surgiram problemas práticos, como guardar sessões em PostgreSQL, recuperar estado, escolher rondas a partir da database e validar o fluxo em Docker.

A fase do dataset foi a que mais cresceu em complexidade. Ao passar de 158 para 300, depois 1 000, 6 000 e finalmente 10 975 locais, tive de aceitar que o problema já não era apenas adicionar entradas. Era necessário criar ferramentas, auditar a qualidade, rejeitar dados incompletos, rever imagens e evitar que o frontend ficasse preso a um ficheiro grande.

O multiplayer entrou depois desta estabilização. Isto foi uma escolha consciente. Se tivesse começado pelo multiplayer antes de guardar bem sessões e rondas, teria aumentado muito o risco. Ao deixar essa extensão para a fase final, consegui aproveitar regras já existentes e concentrar a complexidade nova em salas, SignalR e estado partilhado.

3.2. FRONTEND E EXPERIÊNCIA DE JOGO

No frontend, trabalhei com React, TypeScript e Vite. A primeira dificuldade foi construir um fluxo jogável sem esperar por toda a API. Para isso usei uma camada de dados com modo mock e modo API. Esta decisão ajudou no início, mas na fase final tive de garantir que o modo API era o caminho principal.

A interface final ficou organizada em ecrãs de missão, configuração, ronda, mapa, resultado e relatório final. Também acrescentei estados de loading, erros mais claros, sidebar, topbar, tutorial, retoma de sessão solo, seleção de países e componentes reutilizáveis. Nos testes com utilizadores externos percebi que pequenos detalhes de interface, como botões, navegação, mensagens de erro e estados vazios, tinham impacto direto na demonstração.

Na revisão final também melhorei o fluxo de sessão. A configuração de solo e multiplayer passou a reutilizar os mesmos controlos, o jogador pode escolher países, pode ativar a pontuação total durante a ronda e a sessão solo pode ser retomada depois de recarregar a página. Isto reduziu duplicação no frontend e tornou a experiência mais consistente.

Ecrã	Objetivo	Estado final
Início	Apresentar o jogo, permitir iniciar sessão e aceder ao tutorial.	Implementado.
Configuração	Escolher rondas, tempo, países e opção de pontuação total.	Implementado.
Ronda	Mostrar imagem real, briefing, controlos de zoom e acesso ao mapa.	Implementado.
Mapa	Permitir marcar o palpite e confirmar a posição escolhida.	Implementado.
Debrief da ronda	Mostrar local correto, palpite, distância, pontuação e fonte visual.	Implementado.
Resultado final	Consolidar rondas, pontuação total e mapa da missão.	Implementado.
Multiplayer	Criar ou entrar em sala, preparar jogadores e sincronizar rondas.	Extensão implementada.

Tabela 7 - Ecrãs principais e estado final.

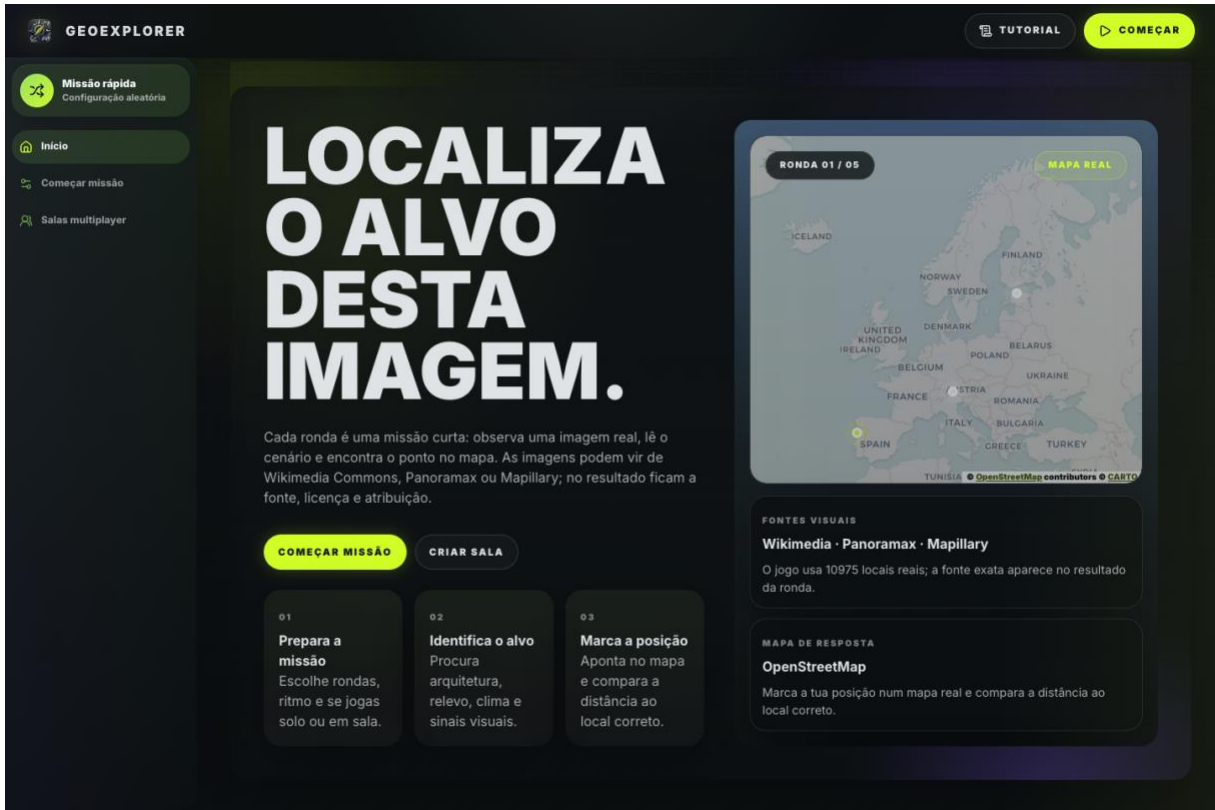


Figura 4 - Página inicial e identidade visual final.

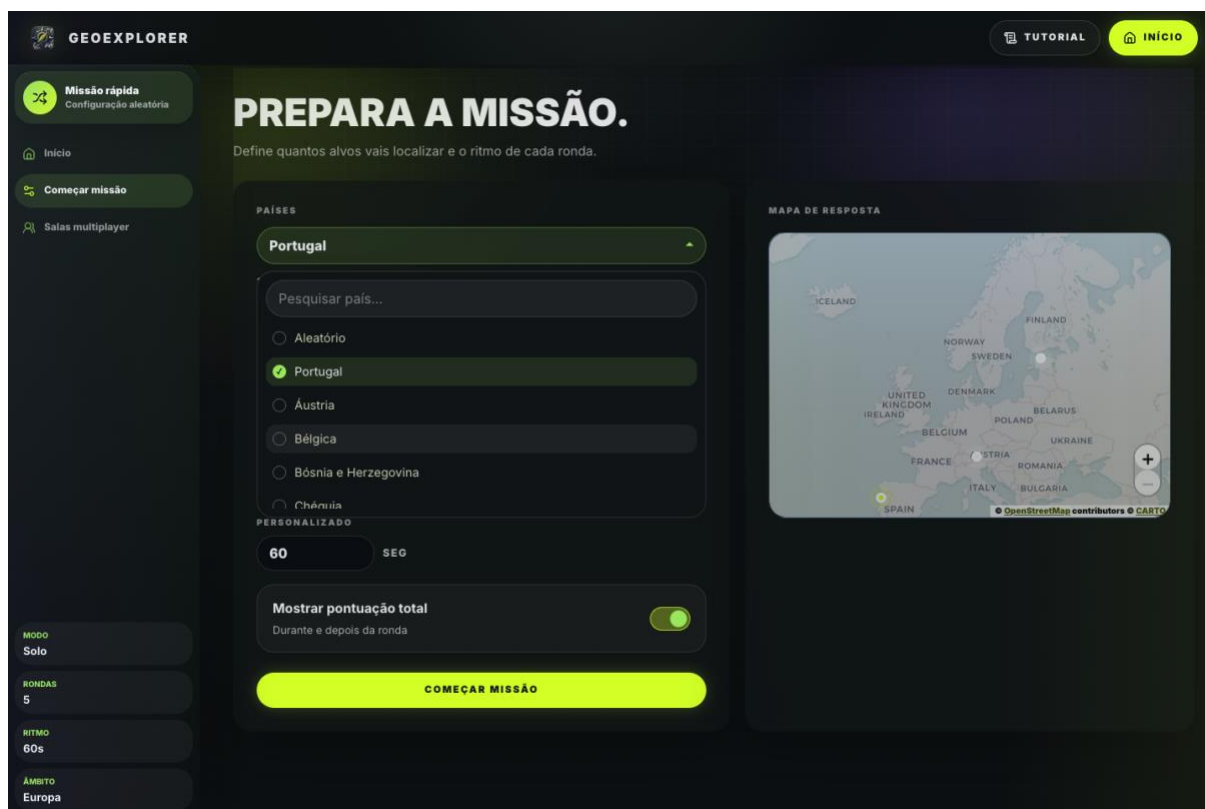


Figura 5 - Configuração de uma missão solo com país, tempo e pontuação total.

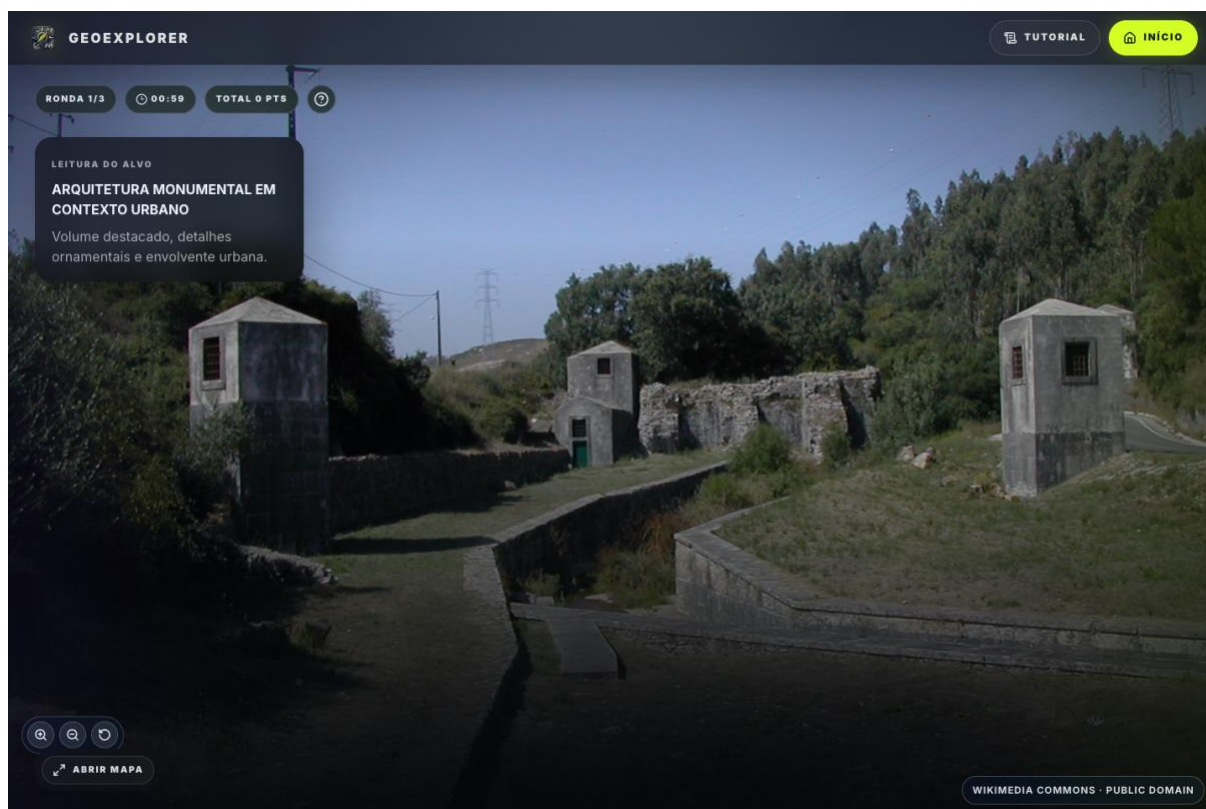


Figura 6 - Ronda com imagem real, briefing e pontuação acumulada.

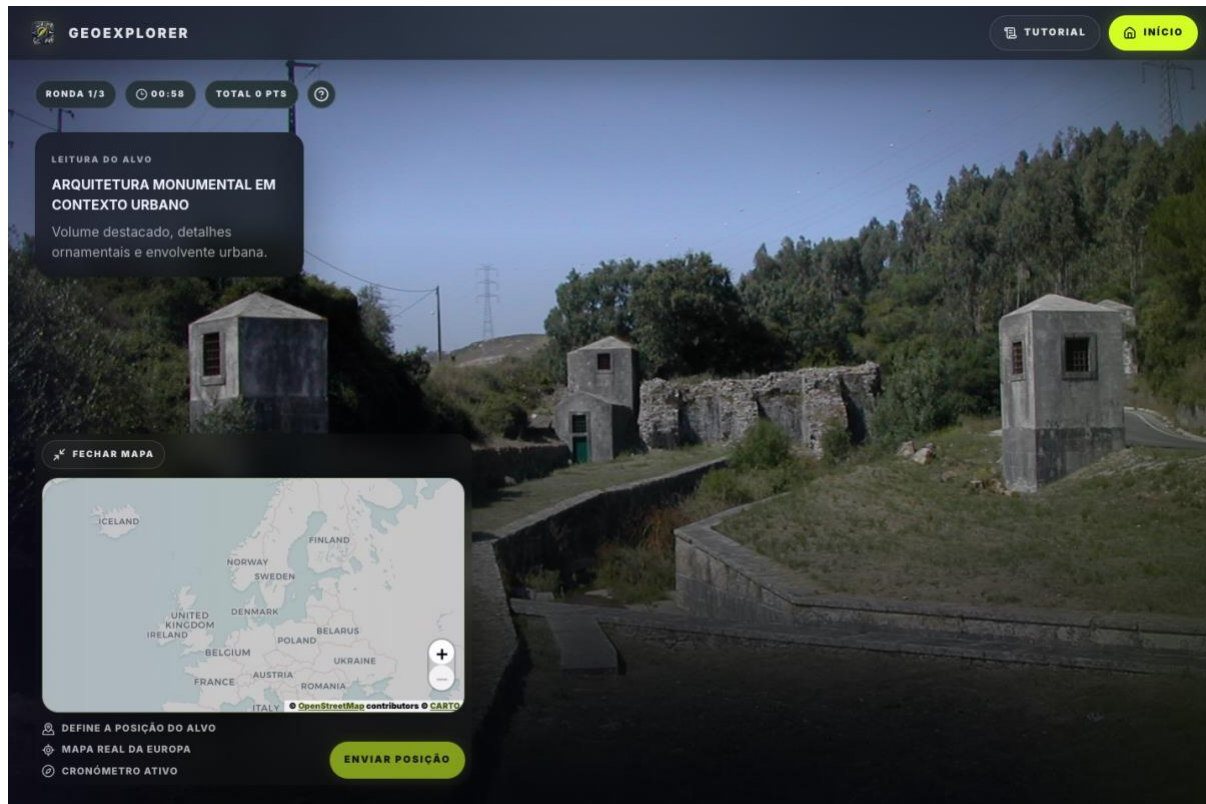


Figura 7 - Mapa real aberto para marcar a posição.

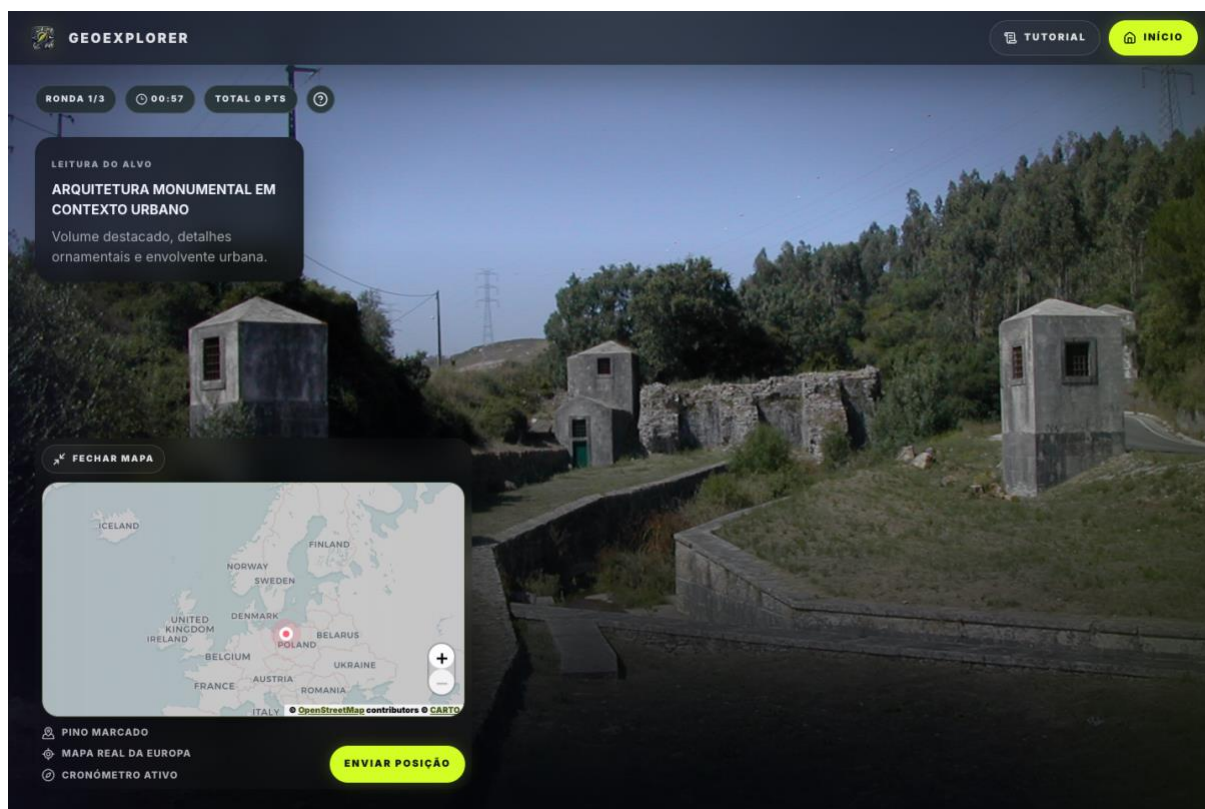


Figura 8 - Palpite marcado antes da submissão.

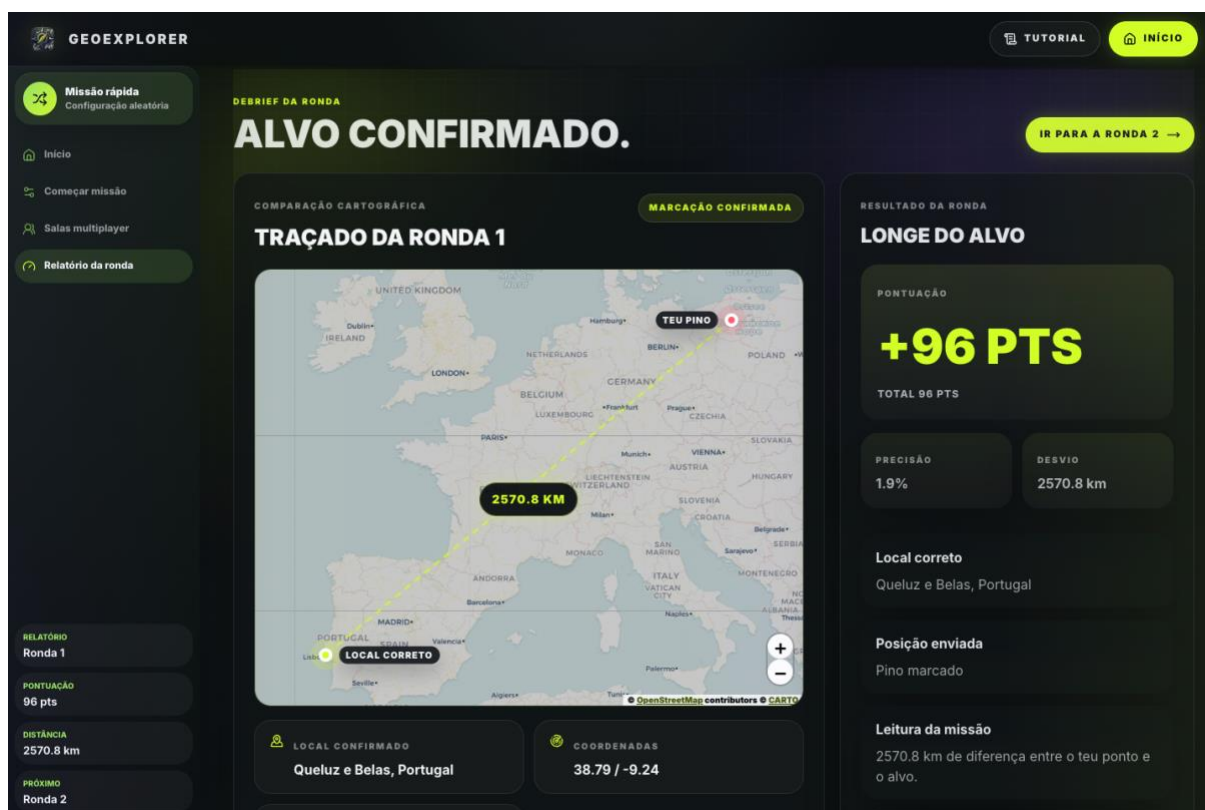


Figura 9 - Debrief da ronda com distância, pontuação da ronda e total acumulado.

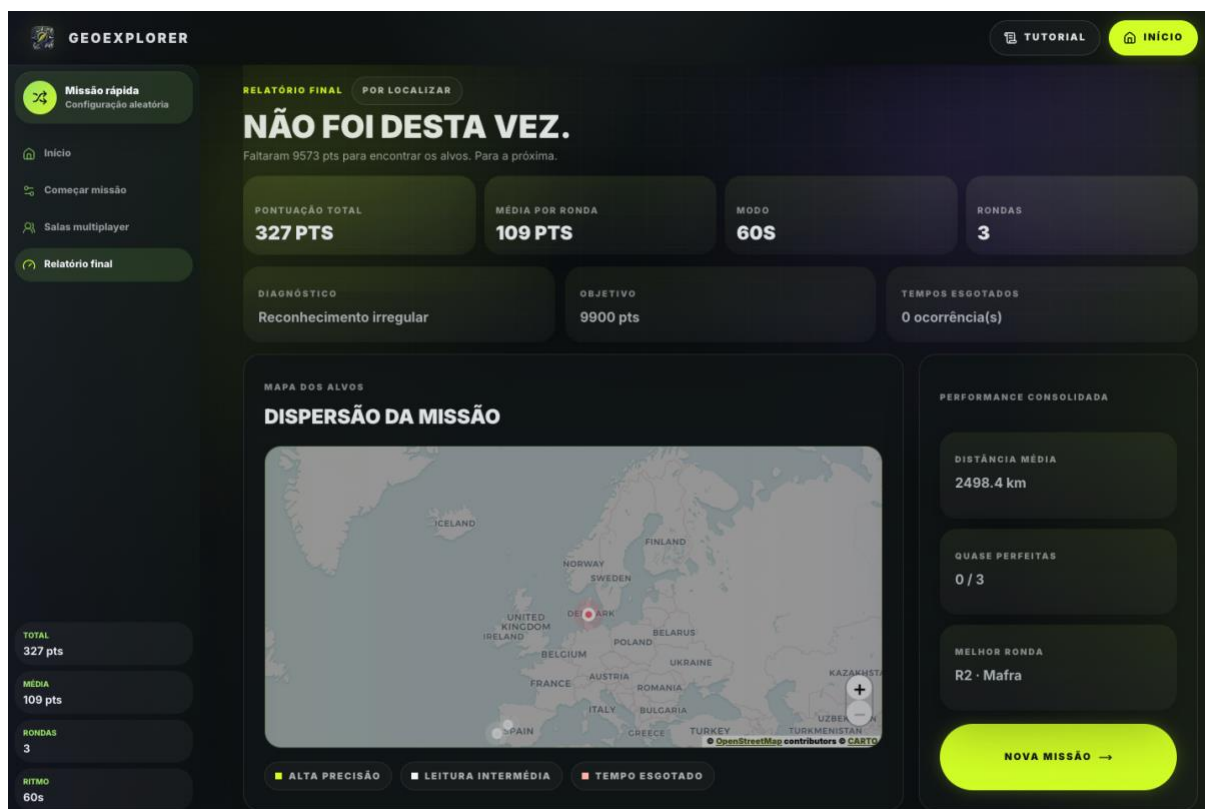


Figura 10 - Resultado final da sessão solo com resumo e mapa da missão.

O frontend foi desenvolvido para funcionar como uma interface de missão. Esta escolha visual ajudou a organizar a informação em blocos claros. O jogador vê o estado da ronda, o tempo, o briefing, a imagem, o mapa e os botões principais sem precisar de navegar por menus complexos.

A evolução visual não se limitou a cores e botões. Houve trabalho para tornar o fluxo mais legível. A configuração passou a explicar melhor a missão, a ronda passou a mostrar informação essencial sem ocupar demasiado espaço e o resultado passou a destacar distância, pontuação e total acumulado. Estas decisões melhoram a compreensão do jogo durante a demonstração.

O mapa expansível foi uma melhoria importante. Num jogo de geolocalização, o mapa precisa de espaço suficiente para decisões precisas. Ao mesmo tempo, a imagem continua a ser o elemento principal durante a observação. A solução final permite alternar entre leitura visual e marcação geográfica sem criar um ecrã demasiado pesado.

3.3 BACKEND E REGRAS DE JOGO

O backend expõe endpoints para criar sessão, obter ronda atual, submeter palpite, resolver timeout e obter resultados. Concentrei as regras de jogo no backend para não depender apenas da UI. Isto inclui validação de coordenadas dentro do mapa europeu, cálculo de distância, pontuação e consistência do estado da ronda.

A distância usa a fórmula de Haversine. A pontuação é maior quando o jogador fica mais perto do local correto. A validação no backend foi importante porque o frontend pode impedir submissões erradas, mas o backend deve continuar a ser a fonte de verdade.

Uma dificuldade que tive foi separar o que pertence à experiência visual do que pertence à regra de jogo. Por exemplo, o frontend pode mostrar melhor ou pior uma mensagem, mas a regra de que só existe um palpite por ronda tem de estar protegida no backend. Esta separação tornou o projeto mais fiável para demonstração.

O backend também passou a proteger detalhes das fontes externas. O endpoint de media para Mapillary evita expor token no frontend e permite ao servidor decidir como resolver a imagem. Este desenho torna o frontend mais estável e dá margem para trocar a forma de obter imagens sem alterar a interface.

A API ficou orientada ao fluxo do jogo, não apenas a operações CRUD. Isto é importante porque uma ronda não é só uma linha numa tabela. Tem estado, regras, resultado e relação com uma sessão. Modelar endpoints em torno do fluxo tornou o código mais próximo da experiência real do utilizador.

3.4 DATABASE, SCHEMA E PERSISTÊNCIA

A database começou como uma intenção documentada e passou a ser parte central do projeto. Quando as flags `GeoExplorer__UsePostgresCatalog` e `GeoExplorer__UsePostgresPersistence` estão ativas, o backend usa o catálogo em PostgreSQL, seleciona candidatos a partir da tabela `locations` e guarda sessões, rondas, palpites e multiplayer.

Na fase final ajustei o carregamento do catálogo para ser mais fiável. Se a tabela `locations` já tiver dados, o backend carrega esses locais diretamente de PostgreSQL, se estiver vazia, recorre ao seed JSON, importa o catálogo e passa a trabalhar sobre a database. Isto evita reimportações desnecessárias e torna o arranque mais previsível em ambientes já inicializados.

O maior ajuste foi perceber que o catálogo completo não devia estar no browser. Com 10 975 locais, faria pouco sentido carregar todo o catálogo no frontend. Ao mover a seleção de rondas para o backend, o frontend recebe apenas os dados necessários para jogar.

As migrations do Entity Framework também tornaram o projeto mais fácil de explicar. Antes, o ficheiro SQL era útil para mostrar intenção, mas podia ficar desatualizado.

Com migrations, o schema nasce do código e pode ser recriado num volume limpo. Isto foi essencial para a validação Docker, porque a avaliação deve conseguir reproduzir a database sem passos escondidos.

O ponto que ficou documentado como cuidado operacional são volumes antigos de PostgreSQL. Se um volume tiver tabelas antigas sem histórico de migrations, o arranque pode encontrar conflitos. Para desenvolvimento, a solução mais simples é recriar o volume. Para produção real, teria de planear uma migração manual mais conservadora.

Uma hosted database não foi adotada na entrega final porque o objetivo era demonstrar o sistema de forma controlada. Para produção, faria sentido avaliar PostgreSQL gerido, backups, métricas, limites de ligação e política de retenção. Para a UC, Docker com PostgreSQL local é suficiente para reproduzir o funcionamento principal.

3.5 EXPANSÃO E AUDITORIA DO DATASET

A expansão do dataset exigiu mais disciplina do que previa. Não bastava encontrar imagens, era necessário garantir origem, licença, atribuição, coordenadas, evitar duplicados fortes, locais demasiado próximos e pistas que revelassem diretamente cidade ou país.

Usei ferramentas locais para procurar candidatos e auditar o conjunto. Essas ferramentas ajudaram a apontar problemas, mas não substituíram a revisão manual. Em alguns casos rejeitei candidatos com dados incompletos ou imagem fraca, mesmo quando isso atrasava o crescimento do dataset.

O objetivo mínimo passou a ser ter cobertura suficiente por país europeu. A maioria dos países ficou com pelo menos 200 locais, o que melhora a experiência quando o filtro por país está ativo. São Marino ficou abaixo desse valor por ser um caso geográfico pequeno, mas permanece registado no dataset e não bloqueia o funcionamento da funcionalidade.

A auditoria ajudou a separar erros bloqueantes de avisos úteis. Erros como coordenadas inválidas, campos obrigatórios em falta ou fontes sem licença seriam graves. Avisos sobre textos parecidos ou entradas a rever podem ficar documentados para melhoria futura. Esta distinção evita parar a entrega por problemas que não impedem o funcionamento.

O aumento para 10 975 locais também justificou a mudança de arquitetura. Carregar todo o catálogo no browser seria pouco eficiente e tornaria a aplicação mais pesada. Ao mover a seleção de rondas para o backend, o frontend recebe apenas a informação necessária para a ronda atual.

A existência de várias fontes visuais obriga a lógica de seleção mais cuidadosa. Nem todas as imagens têm o mesmo formato, a mesma licença ou o mesmo comportamento

de URL. Por isso, a aplicação trata fontes principais jogáveis e fontes opcionais de forma diferente. Esta decisão reduz surpresas durante a demonstração.

3.6 MULTIPLAYER

O multiplayer foi acrescentado depois de o modo solo estar estável. Esta ordem foi importante para controlar o risco. O dono cria a sala, escolhe configuração e inicia a partida. Os restantes jogadores entram por link ou lista pública. O resultado da ronda aparece quando todos os jogadores ativos submetem ou quando o tempo termina.

Tive de resolver casos de dono da sala, nomes únicos, salas públicas, password opcional, refresh/reconnect e jogadores que saem durante a partida. O backend guarda salas, jogadores, rondas e palpites em PostgreSQL quando a persistência está ativa. As passwords das salas não ficam em texto simples, o backend guarda apenas hash.

No fecho do frontend, também ajustei o resultado multiplayer para mostrar melhor a pontuação da ronda, o total acumulado e os palpites dos outros jogadores no mapa quando existem dados suficientes. Isto ajuda a perceber rapidamente quem ficou mais perto do alvo.

The screenshot displays the 'MULTIPLAYER' section of the GEOEXPLORER application. At the top, there's a navigation bar with a menu icon, the 'GEOEXPLORER' logo, and buttons for 'TUTORIAL' and 'INÍCIO'. The main heading reads 'MONTA UMA EQUIPA OU ENTRA POR CONVITE.' Below this, the interface is split into two columns. The left column, titled 'SALA', contains a 'NOME NA SALA' field with the text 'Rapid Orbit 772', a 'VISIBILIDADE' section with 'POR LINK' and 'SALA ABERTA' buttons, and a 'PASSWORD OPCIONAL' field with the placeholder 'Deixa em branco para acesso direto'. A large yellow 'CRIAR SALA' button is at the bottom of this column. The right column, titled 'CONVITE', has the heading 'ENTRAR NUMA SALA EXISTENTE', a subtext 'Usa o código recebido por link. O nome tem de ser único dentro da sala.', a 'CÓDIGO DA SALA' field with 'ABCD12', a 'PASSWORD, SE EXISTIR' field with 'Opcional', and a large yellow 'ENTRAR NA SALA' button. Below these columns is a section titled 'SALAS ABERTAS' with the heading 'ESCOLHER SALA ABERTA'. It includes a subtext 'Salas no lobby sincronizadas em tempo real.', buttons for '0 salas' and 'Tempo real', a search bar with the placeholder 'Pesquisar por código, dono ou ritmo', and a message 'Não há equipas abertas neste momento. Cria uma equipa aberta ou entra por link.'

Figura 11 - Entrada multiplayer com criação ou entrada em sala.

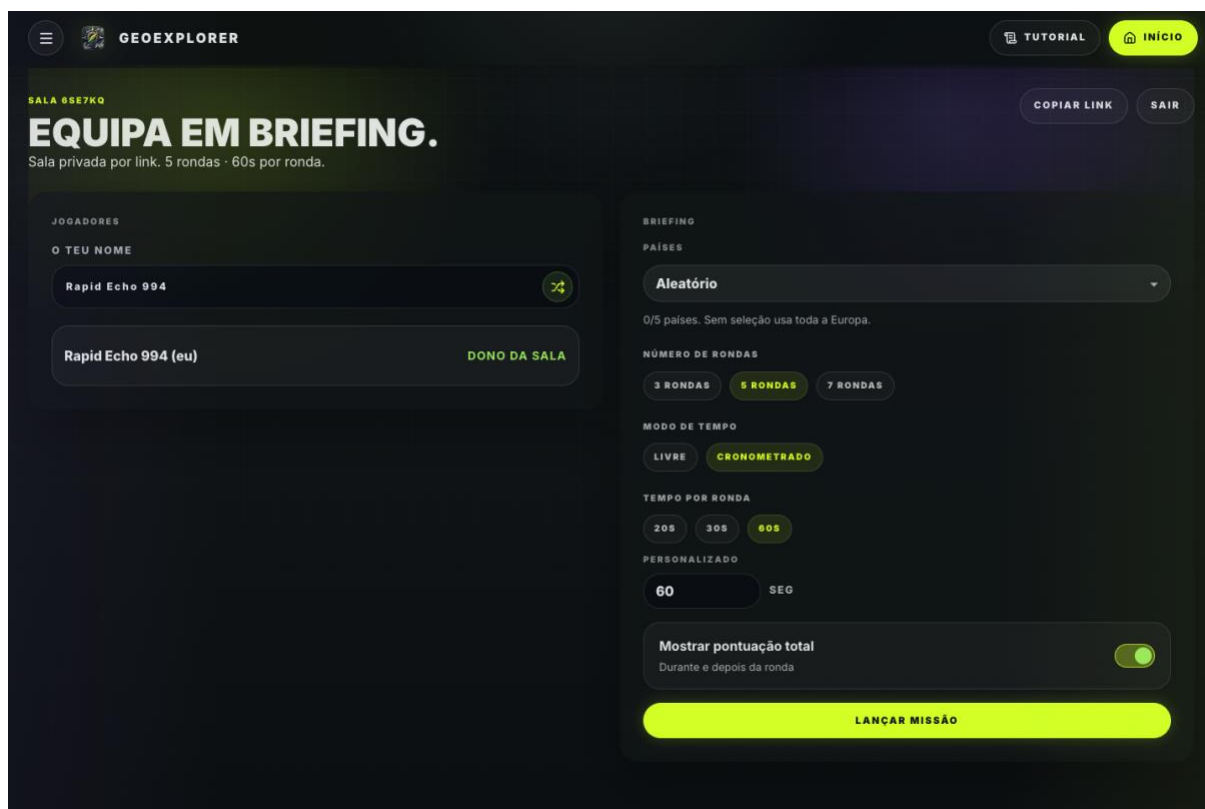


Figura 12 - Lobby multiplayer com jogadores, país, tempo e pontuação total.

A funcionalidade ficou limitada a uma escala de demonstração. Numa instância única, o estado em memória é suficiente para validar salas e interação. Para escala horizontal, seria necessário sincronizar estado entre instâncias, por exemplo com Redis backplane ou persistência adicional. Esta limitação está assumida e não contradiz o objetivo final.

A decisão de não juntar multiplayer com contas de utilizador manteve a implementação controlada. Contas mudariam o tipo de problema, porque passariam a envolver autenticação, recuperação, histórico e proteção de dados pessoais, neste projeto, salas temporárias eram suficientes e mais coerentes com o âmbito.

3.7 DOCKER, LOGS E VERSÃO PÚBLICA

O Docker Compose ficou com perfis para frontend mock, database isolada e perfil full. O perfil full arranca frontend em modo API, backend e PostgreSQL. Também configurei portas por variáveis de ambiente, CORS configurável e logs técnicos com Serilog.

Também adicionei `/api/health/details` para confirmar no servidor se a database responde e se uma imagem Mapillary consegue ser resolvida pelo backend. Fiz isto porque, durante os testes na VPS, por vezes o Mapillary não parecia estar a funcionar e o health simples não chegava para perceber se o problema estava na aplicação, no token ou na resposta externa.

A versão pública em <https://geoexplorer.firmwork.pt/> foi usada para testar fora do ambiente local. A VPS permitiu partilhar o jogo com utilizadores externos, incluindo pessoas próximas, abrir salas em paralelo e observar problemas que no computador de desenvolvimento eram menos visíveis.

Docker Compose teve dois papéis no projeto. Durante o desenvolvimento, ajudou a levantar dependências como PostgreSQL. Na validação final, permitiu executar o perfil full com frontend, backend e database em conjunto. Esta segunda parte é importante porque demonstra integração, não apenas componentes isolados.

O perfil full também obrigou a rever variáveis de ambiente e comunicação entre serviços. O frontend precisa de saber que está em modo API e o backend precisa de ligação à database. Pequenas diferenças de configuração podem quebrar a demonstração, por isso a validação Docker foi incluída como evidência final.

Os logs foram úteis para diagnosticar problemas de catálogo e Mapillary. Quando uma fonte externa não responde como esperado, o erro não deve parecer uma falha invisível do frontend. O backend passou a registar situações relevantes e a permitir perceber se está a usar database, seed JSON ou fallback.

A versão pública foi tratada como apoio de validação e não como garantia de produção. Serviu para testar fora do computador local, partilhar com outras pessoas e observar problemas reais de rede, browser e dispositivo. Ainda assim, o deployment continua experimental e não substitui uma infraestrutura preparada para operação contínua.

3.8 APOIO DE IA

Ferramenta	Uso	Validação feita por mim
ChatGPT	Planeamento, estrutura, revisões/melhorias de código, revisão de textos, apoio em testes e ideias para ferramentas de dataset.	Revisei decisões no código, nos testes, no Docker e nos textos.
Gemini	Exploração visual inicial do logótipo.	Integrei e ajustei a identidade visual no frontend.
Ferramentas locais geradas/adaptadas	Auditorias e expansão do dataset.	Não substituíram revisão manual nem validação técnica.

Tabela 8 - Uso de IA e responsabilidade final.

Usei IA como apoio, não como substituição da validação. O ChatGPT ajudou-me a estruturar documentação, rever problemas técnicos, pensar em testes e sugerir ferramentas para lidar com o dataset. O Gemini foi usado para explorar ideias visuais do logótipo. As decisões finais ficaram sempre dependentes do código, testes, Docker e revisão manual.

O apoio de IA foi usado como ferramenta de trabalho, não como substituto da responsabilidade técnica. Serviu para rever texto, propor estrutura, acelerar tarefas repetitivas e ajudar a encontrar incoerências entre relatório, documentação e código. As decisões finais, validação e aceitação das alterações permaneceram responsabilidade do autor do projeto.

Esta utilização também teve limites. Texto gerado automaticamente pode soar estranho, repetir construções ou introduzir afirmações demasiado genéricas. Por isso, a revisão final procurou manter linguagem corrente em português de Portugal, evitar formulações artificiais e garantir que cada afirmação correspondia a algo existente no projeto.

4. Testes e Validação

Este capítulo reúne a validação feita sobre a aplicação, o dataset, o Docker, a versão pública e o próprio relatório. Usei testes automáticos e validação manual porque nem todos os riscos do projeto aparecem em testes unitários.

4.1 ESTRATÉGIA DE VALIDAÇÃO

A validação final teve várias camadas. Não confiei apenas em testes automáticos nem apenas em demonstração manual. Usei testes backend, testes frontend, build, auditoria do dataset, validação Docker, revisão de dependências e testes com a versão pública.

Esta combinação foi importante porque cada camada apanha problemas diferentes. Os testes unitários e de integração validam regras. O build confirma que o frontend gera a versão de produção sem erros. A auditoria encontra problemas no dataset. Docker confirma execução reproduzível. A VPS e os testes com utilizadores externos mostram problemas de experiência real.

A estratégia também acompanhou a evolução do projeto. Enquanto o frontend usava mock, bastava validar experiência e navegação. Depois da passagem para API + PostgreSQL, tornou-se necessário testar endpoints, persistência, migrations e integração. Com multiplayer, entraram cenários de sala, presença e atualização em tempo real.

A validação não tentou provar que a aplicação suporta produção em larga escala. Esse objetivo seria desproporcionado para a UC. O que foi validado é que o fluxo principal funciona, que os dados estão estruturados, que o sistema executa em ambiente reproduzível e que as principais limitações estão documentadas.

4.2 VALIDAÇÃO AUTOMÁTICA

Verificação	Resultado	Leitura
dotnet test GeoExplorer.slnx	98 testes passados	Cobriu serviços, API, persistência, catálogo, diagnostics e regras multiplayer.
npm test -- --run	98 testes passados	Cobriu fluxo frontend, componentes, sidebar, mapa, multiplayer e comunicação com a API.
npm run build	Build concluído	A divisão de chunks removeu o aviso anterior de bundle superior a 500 kB.
audit-location-dataset --fail-on-errors	10 975 locais sem erros bloqueantes	Ainda há avisos úteis para revisão manual futura.
docker compose --profile full config	Configuração válida	Frontend, backend e PostgreSQL alinhados.
dotnet list package --vulnerable	Sem vulnerabilidades NuGet conhecidas	Resultado limpo para backend e testes backend.
npm audit	6 vulnerabilidades altas em dev dependencies	Ligadas a esbuild/Vite/Vitest, correção automática exige atualização breaking.

Tabela 9 - Validação automática realizada.

Registei também os pontos que não estão totalmente fechados. O npm audit aponta vulnerabilidades altas em dependências de desenvolvimento, ligadas à cadeia esbuild/Vite/Vitest. A correção automática exige atualização breaking, por isso preferi documentar o risco em vez de fazer uma atualização apressada perto da entrega.

Os testes backend foram importantes para regras de sessão, persistência e multiplayer. Nesta fase, o risco já não era apenas um endpoint devolver uma resposta, era garantir que estados como ronda resolvida, sala concluída, jogador desconectado e resultado final continuavam consistentes.

Os testes frontend ajudaram a manter a comunicação com a API e os componentes estáveis. Como o frontend tem modo mock e modo API, os testes reduzem o risco de uma alteração numa camada partir a outra. Mesmo assim, mantive validação manual porque a experiência de mapa, imagens e multiplayer não é totalmente capturada por testes unitários.

A validação automática também inclui build. Um teste pode passar e, mesmo assim, o projeto falhar ao gerar a versão de produção. O comando de build confirma que o frontend compila sem erros de TypeScript ou importações quebradas. Este passo foi mantido como parte da verificação final.

A auditoria do dataset foi tratada como teste do produto. Num jogo baseado em locais reais, dados inválidos são falhas funcionais. Por isso, a verificação de coordenadas, fontes, licenças e estrutura do catálogo tem o mesmo peso prático que um teste de código.

A lista de dependências vulneráveis também foi revista. Algumas vulnerabilidades estavam associadas a dev dependencies do ecossistema frontend. Foram documentadas como risco de revisão, sem afirmar que a aplicação estava pronta para produção. Esta abordagem é mais honesta do que esconder avisos ou tratá-los como bloqueadores sem contexto.

4.3 VALIDAÇÃO MANUAL

Cenário	Resultado	Observação
Docker full com volume limpo	Validado	Migrations aplicadas, catálogo carregado/importado e serviços a responder.
Sessão solo curta	Validada	Criação, palpite, resultado de ronda e relatório final.
Sala multiplayer curta	Validada	Dois jogadores, ronda sincronizada, palpites e resultado.
VPS / versão pública	Validada	Health check, verificação completa da API, diagnóstico da database, criação de sala e testes com utilizadores externos.
Screenshots finais	Validadas visualmente	Capturas refeitas localmente com solo em modo mock e multiplayer em modo API com backend local.

Tabela 10 - Validação manual e operacional.

Na validação Docker limpa, confirmei que as migrations InitialCreate, AddMultiplayerRooms e AddMultiplayerRoomAccess eram aplicadas. Depois joguei uma sessão solo curta e uma sala multiplayer curta. No fim, a database tinha catálogo, sessão, ronda, sala, jogadores e palpites esperados.

A validação manual começou pelo fluxo solo. Foram verificados início de sessão, configuração de rondas, escolha de país, modo cronometrado, abertura do mapa, submissão de palpite, resultado de ronda e resultado final. Esta sequência é a experiência principal do utilizador e precisava de funcionar sem passos ocultos.

A seguir foi verificado o comportamento visual. O objetivo foi confirmar que os textos cabiam nos componentes, que os botões principais estavam acessíveis, que o mapa não escondia informação essencial e que as imagens carregavam de forma consistente. Esta revisão foi necessária porque o frontend mudou bastante perto da fase final.

Os testes com utilizadores externos deram uma perspetiva diferente. Pessoas fora do ambiente de desenvolvimento tendem a clicar em sequências menos previsíveis, usar tamanhos de ecrã diferentes e interpretar textos de forma mais direta. Esse contacto ajudou a melhorar mensagens, fluxo multiplayer e clareza dos ecrãs.

A versão pública foi especialmente útil para testar problemas que não aparecem localmente. Latência, cache do browser, configuração de CORS, caminhos de API e carregamento de imagens podem comportar-se de forma diferente fora do ambiente de desenvolvimento. A validação em VPS ajudou a tornar a demonstração mais robusta.

A validação manual também serviu para confirmar a coerência do relatório. As imagens usadas no documento foram atualizadas para corresponder ao frontend final. Isto evita uma falha comum em relatórios de projeto, onde a aplicação evolui mas as figuras continuam a mostrar versões antigas.

4.4 AUDITORIA DO DATASET

A auditoria do dataset verifica IDs duplicados, imagens repetidas, dados obrigatórios em falta, locais demasiado próximos, textos repetidos, pistas diretas e possíveis imagens aéreas. Esta etapa foi importante porque um dataset grande só é útil se mantiver qualidade mínima.

Mesmo com auditoria limpa para erros bloqueantes, mantive avisos como material de revisão futura. Alguns textos parecidos em grupos grandes não impedem a entrega, mas são úteis para continuar a melhorar a experiência depois da UC.

O resultado mais importante foi não existirem erros bloqueantes. Isto significa que o catálogo pode ser usado pela aplicação sem falhas estruturais conhecidas. Os avisos que ficaram são úteis para revisão futura, mas não impedem a entrega nem a demonstração do fluxo principal.

A distribuição por país também foi revista. A maioria dos países europeus ficou com cobertura suficiente para suportar o filtro por país. A exceção principal é São Marino, que ficou abaixo dos 200 locais por ser um território pequeno e com menor disponibilidade prática de fontes. Esta exceção deve ser explicada como limitação específica e não como falha da funcionalidade.

A auditoria também reforça a diferença entre quantidade e qualidade. Um dataset grande só é útil se puder ser confiável. Por isso, a validação automática ficou associada à origem, licença e coordenadas, não apenas ao número final de entradas.

4.5 RISCOS E SEGURANÇA

Risco	Estado	Mitigação
Dataset inconsistente ou com licenças fracas.	Mitigado para MVP	Auditoria local, metadados obrigatórios e revisão de imagens fracas.
Carregar o catálogo completo no frontend.	Mitigado	API + PostgreSQL passaram a ser o fluxo real.
Dependência excessiva de APIs externas.	Mitigado parcialmente	Dataset local e Mapillary opcional resolvido pelo backend.
Complexidade do multiplayer.	Assumido	Implementação incremental, SignalR e testes de sala curta.
Segredos expostos.	Mitigado	.env fora do Git, token Mapillary só no ambiente e CORS configurável.
Vulnerabilidades em dev dependencies.	Documentado	Registado no relatório, atualização exige cuidado por poder ser breaking.

Tabela 11 - Riscos finais e mitigação.

A parte de segurança foi tratada de forma proporcional ao projeto. Não implementei autenticação completa nem um sistema de permissões avançado, mas tratei os riscos reais da entrega, como segredos, CORS, validação de inputs, password de sala, logs e dependências.

A validação por critérios de aceitação também ficou coerente com a proposta. Consegui iniciar sessão, configurar rondas, jogar com mapa real, submeter palpite, calcular distância e pontuação, mostrar resultado de ronda, apresentar relatório final e guardar dados. A extensão multiplayer não substitui estes critérios, acrescenta um cenário adicional já depois do núcleo estar fechado.

A principal fragilidade assumida continua a ser a preparação para produção. O projeto está preparado para demonstração, mas não afirmo que está pronto para uma escala pública sem mais trabalho. No caso do multiplayer, SignalR funciona bem numa instância, mas uma instalação com várias instâncias exigiria sincronização de mensagens e estado, por exemplo com Redis backplane ou solução equivalente. Também faltariam load testing, observabilidade mais completa, política de retenção, autenticação se houver histórico pessoal e revisão cuidada das dependências Node.

4.6 VERIFICAÇÃO DO RELATÓRIO

Para este relatório final, também fiz validação visual do próprio DOCX. O objetivo é evitar que o relatório pareça mais fraco que o intercalar. Por isso reconstruí a estrutura com capa no estilo do intercalar, índice/listas, capítulos mais desenvolvidos, figuras finais, tabelas de validação e anexos técnicos reais.

5. Conclusões

Este capítulo fecha o relatório com o resultado final, as limitações assumidas, o trabalho futuro e uma reflexão pessoal sobre as decisões tomadas durante o projeto.

5.1 RESULTADO FINAL

O resultado final foi além do MVP inicial no volume e na qualidade do dataset e no multiplayer. Mesmo assim, a prioridade continuou a ser entregar uma aplicação jogável, demonstrável e com funcionamento técnico claro. O jogador consegue observar uma imagem real, marcar uma posição no mapa, receber pontuação e consultar resultados. O backend consegue guardar estado essencial em PostgreSQL. A arquitetura suporta uma extensão multiplayer com SignalR.

Comparando com os resultados esperados no relatório intercalar, realizei os pontos principais previstos. A aplicação ficou com sessão de jogo completa com várias rondas europeias, modo livre e modo cronometrado, mapa real para seleção e comparação do palpite, gravação de sessões, rondas, palpites e resultados, dataset expandido com origem, licença e atribuição, testes automáticos e relatório final coerente com o código e com o histórico do projeto.

A maior aprendizagem técnica foi perceber quando uma solução que ajudou no início deixa de ser adequada. O modo mock foi essencial para construir o frontend rapidamente, mas deixou de ser o caminho certo quando o catálogo cresceu. Essa mudança para API + PostgreSQL foi uma decisão de consolidação do projeto.

O resultado final também pode ser avaliado pela coerência entre o que foi proposto, o que foi construído e o que ficou documentado. O projeto não ficou apenas com uma interface visual. Existe backend, database, dataset auditado, Docker, testes, imagens atualizadas no relatório e explicação das decisões técnicas.

A demonstração ficou mais robusta porque o fluxo real está suportado pela API. O modo mock continua útil para desenvolvimento, mas deixou de ser a principal prova do projeto. Esta passagem mostra consolidação do projeto, porque uma aplicação com dados reais e persistência obriga a resolver problemas que não aparecem num protótipo isolado.

5.2 LIMITAÇÕES

Limitação	Motivo	Evolução possível
Sem contas de utilizador	Não era necessário para demonstrar o MVP.	Adicionar autenticação e histórico pessoal.
Europa como âmbito	Reduziu risco de dados e tornou a demo mais controlada.	Expandir depois de estabilizar fontes e licenças.
Mapillary opcional	Depende de token e miniaturas temporárias.	Resolver sempre pelo backend ou trocar por fonte aberta suficiente.
Multiplayer sem escala real comprovada	Foi extensão final, não núcleo inicial.	Mais testes com vários jogadores e eventual Redis backplane.
Deployment público ainda experimental	A UC avalia sobretudo projeto, código e relatório.	Deployment gerido, observabilidade e database hosted.

Tabela 12 - Limitações finais.

A principal limitação é a escala. O GeoExplorer foi validado para demonstração, testes locais, versão pública experimental e utilização controlada. Não foi realizado load testing suficiente para afirmar comportamento com muitos jogadores, várias salas concorrentes ou uso contínuo durante longos períodos.

A ausência de contas de utilizador também limita a evolução do produto. Sem contas, não há histórico pessoal, rankings persistentes, recuperação de progresso ou moderação por perfil. Esta escolha foi correta para o MVP, mas teria de mudar se a aplicação passasse a ter utilização pública recorrente.

O dataset é grande para o contexto do projeto, mas continua a exigir revisão futura. Algumas imagens podem ser menos interessantes para o jogo, algumas fontes podem mudar e alguns países têm melhor cobertura do que outros. A auditoria estrutural reduz erros, mas não substitui curadoria visual contínua.

O deployment público é experimental. A VPS ajudou a validar o jogo fora do ambiente local, mas não representa uma infraestrutura de produção. Faltam observabilidade, alertas, backups, política de retenção, gestão formal de segredos e rotina de atualização de dependências.

Estas limitações não reduzem o cumprimento do objetivo académico. Pelo contrário, tornam mais clara a fronteira entre aplicação demonstrável e produto comercial. O projeto mostra que o núcleo funciona e identifica o que seria necessário antes de assumir uma utilização mais ampla.

5.3 TRABALHO FUTURO

Se o projeto continuasse, seguiria uma ordem conservadora. Primeiro viriam contas de utilizador e histórico pessoal, depois testes multiplayer com mais jogadores e reconexão mais robusta, depois deployment público mais estável, e só depois avaliaria PostgreSQL hosted, Supabase ou Turso com métricas reais.

- Adicionar autenticação e histórico por utilizador se for necessário guardar progresso pessoal.
- Melhorar multiplayer com load testing, reconexão e eventual Redis backplane.
- Continuar a rever imagens, licenças e fontes visuais.
- Avaliar hosted database com base em leituras/escritas reais.
- Preparar observabilidade e políticas de retenção se a aplicação for publicada a longo prazo.

O primeiro passo futuro deveria ser consolidar contas de utilizador apenas se existisse necessidade real de histórico pessoal. Esta funcionalidade teria impacto em database, privacidade, segurança e interface. Por isso, não deve ser vista como simples ecrã de login, mas como alteração de âmbito.

O segundo passo deveria ser reforçar o multiplayer com testes mais exigentes. Seria necessário medir várias salas em simultâneo, perda de ligação, reconexão, jogadores atrasados e comportamento com múltiplas instâncias. Só depois faria sentido decidir se Redis backplane ou outra abordagem é necessária.

O terceiro passo seria preparar uma operação pública mais estável. Isso inclui hosted database, backups, logs centralizados, métricas, alertas, gestão de segredos e política de retenção. Estas tarefas não tornam o jogo mais visível para o utilizador, mas são essenciais para manter o sistema em funcionamento.

O quarto passo seria continuar a curadoria do dataset. Mais locais não significam automaticamente melhor experiência. A melhoria futura deveria combinar quantidade, qualidade visual, equilíbrio por país e remoção de imagens repetitivas ou pouco úteis para treino de geolocalização.

5.4 REFLEXÃO PESSOAL

Este projeto obrigou-me a passar por várias fases, desde a proposta e o frontend jogável até à API, database, Docker, dataset, multiplayer, validação e relatório. Tive dificuldades principalmente no crescimento do dataset, na integração com PostgreSQL/migrations, na clareza do multiplayer e na diferença entre uma app que funciona localmente e uma app que é demonstrável noutra máquina.

A parte mais importante foi transformar problemas em decisões documentadas. Quando percebi que carregar todo o catálogo no browser era uma má ideia, mudei a arquitetura. Quando percebi que Mapillary tinha URLs temporárias e por vezes não parecia responder no servidor, criei endpoints no backend para resolver miniaturas e verificar o estado real. Quando o multiplayer podia fugir ao âmbito, mantive-o como extensão controlada. Estas decisões tornaram o projeto mais defendível.

O feedback exterior também teve peso. Quando utilizadores externos testaram a aplicação, percebi que a parte técnica podia estar correta e, mesmo assim, a experiência continuar pouco clara. Isso levou-me a rever textos, estados de erro, botões, entrada em salas, visibilidade do resultado e forma como o jogo comunica progresso ao jogador.

Houve ponderações que ficaram pelo caminho. Pensei em autenticação, ranking, cobertura global, fontes visuais obrigatórias em 360 e uma vertente multiplayer mais ambiciosa. Acabei por limitar essas ideias porque aumentariam muito o risco perto da entrega. Prefiri entregar uma base funcional, validada e fácil de explicar, deixando essas evoluções como trabalho futuro.

Também aprendi que um relatório final não deve ser apenas um resumo do código. Tem de explicar o caminho, as dificuldades e as escolhas. Por isso, nesta versão final, procurei mostrar mais do que capturas do frontend. Mostro arquitetura, database, dataset, validação, riscos, limitações e excertos técnicos suficientes para o júri perceber como o projeto foi construído.

A maior dificuldade foi equilibrar ambição e entrega. Era fácil imaginar funcionalidades como contas, rankings, cobertura global e modos competitivos. O trabalho mais importante foi perceber quais dessas ideias ajudavam a demonstrar o projeto e quais apenas aumentavam risco. Essa escolha tornou o resultado final mais consistente.

Também ficou claro que dados reais mudam a natureza do projeto. Com mock, quase tudo parece controlado. Com imagens reais, fontes externas, licenças, coordenadas e database, surgem problemas de qualidade, desempenho e validação. Resolver estes pontos deu mais valor técnico ao projeto do que acrescentar ecrãs isolados.

A fase final mostrou ainda a importância de testar fora do ambiente de desenvolvimento. A aplicação funcionava localmente, mas a versão pública e os utilizadores externos revelaram detalhes de fluxo e interface que passavam despercebidos. Esse contacto ajudou a tornar a demonstração mais segura.

O relatório também foi parte do trabalho técnico. Rever textos, alinhar imagens, confirmar comandos, explicar limitações e evitar afirmações exageradas ajudou a transformar a aplicação numa entrega defensável. O objetivo final não foi parecer maior do que era, mas mostrar com clareza o que foi realizado e o que ficou conscientemente fora.

Bibliografia

Brown, S. (s.d.). The C4 model for visualising software architecture. Consultado em 2 de maio de 2026, em <https://c4model.com/>

Docker Inc. (s.d.). Docker Compose overview. Consultado em 2 de maio de 2026, em <https://docs.docker.com/compose/>

Leaflet. (s.d.). Leaflet documentation. Consultado em 2 de maio de 2026, em <https://leafletjs.com/reference.html>

Microsoft. (s.d.). ASP.NET Core documentation. Consultado em 2 de maio de 2026, em <https://learn.microsoft.com/aspnet/core/>

Microsoft. (s.d.). Entity Framework Core documentation. Consultado em 2 de maio de 2026, em <https://learn.microsoft.com/ef/core/>

Microsoft. (s.d.). SignalR documentation. Consultado em 28 de maio de 2026, em <https://learn.microsoft.com/aspnet/core/signalr/>

OpenStreetMap contributors. (s.d.). Copyright and license. Consultado em 2 de maio de 2026, em <https://www.openstreetmap.org/copyright>

Panoramax. (s.d.). API documentation. Consultado em 14 de maio de 2026, em <https://docs.panoramax.fr/backend/api/api/>

PostgreSQL Global Development Group. (s.d.). PostgreSQL documentation. Consultado em 2 de maio de 2026, em <https://www.postgresql.org/docs/>

React. (s.d.). React documentation. Consultado em 2 de maio de 2026, em <https://react.dev/>

Vite. (s.d.). Vite guide. Consultado em 2 de maio de 2026, em <https://vite.dev/guide/>

Wikimedia Commons. (s.d.). Reusing content outside Wikimedia. Consultado em 2 de maio de 2026, em https://commons.wikimedia.org/wiki/Commons:Reusing_content_outside_Wikimedia

Mapillary. (s.d.). API documentation. Consultado em 19 de maio de 2026, em <https://www.mapillary.com/developer/api-documentation>

Comissão Nacional de Proteção de Dados. (s.d.). Consentimento. Consultado em 14 de junho de 2026, em <https://www.cnpd.pt/organizacoes/areas-tematicas/consentimento/>

Comissão Nacional de Proteção de Dados. (2021). Nota informativa sobre cookies. Consultado em 14 de junho de 2026, em https://www.cnpd.pt/media/x2zdus5o/nota-informativa-cnpd_cookies_20210625.pdf

European Data Protection Board. (s.d.). EDPB Cookie Policy. Consultado em 14 de junho de 2026, em https://www.edpb.europa.eu/concernant-le-cepd/mentions-legales/edpb-cookie-policy_en

Comissão Europeia. (s.d.). Cookies policy. Consultado em 14 de junho de 2026, em https://commission.europa.eu/cookies-policy_en

Anexo A - Excertos Técnicos Selecionados

Não anexo ficheiros completos. Os excertos seguintes mostram zonas do código que justificam decisões importantes, como comunicação HTTP, validação, carregamento do catálogo, schema, SignalR e Docker.

Excerto	Ficheiro	O que demonstra
A1	src/backend/Program.cs	Registo de serviços, CORS, API, Mapillary e SignalR.
A2	src/frontend/src/services/apiGameDataSource.ts	Comunicação entre frontend e API no modo real.
A3	src/backend/Services/GameRoundRules.cs	Validação de coordenadas, distância e pontuação.
A4	src/backend/Services/SeedLocationCatalog.cs	Carregamento do catálogo PostgreSQL antes do seed JSON.
A5	src/backend/Data/GeoExplorerDbContext.cs	Schema EF para solo, catálogo e multiplayer.
A6	src/backend/Hubs/MultiplayerHub.cs	Envio de eventos em tempo real por SignalR.
A7	docker-compose.yml	Perfil full com frontend, backend e PostgreSQL.

Tabela 13 - Excertos técnicos selecionados.

A1. REGISTO DA API, CORS, DATABASE E SIGNALR

Fonte: src/backend/Program.cs, linhas 16-32.

```
16
17 builder.Services.AddProblemDetails();
18 builder.Services.AddMemoryCache();
19 builder.Services.AddSignalR();
20 builder.Services.AddCors(options =>
21 {
22     var allowedOrigins = GetAllowedOrigins(builder.Configuration);
23     options.AddDefaultPolicy(policy =>
24     {
25         policy
26             .WithOrigins(allowedOrigins)
27             .AllowAnyHeader()
28             .AllowAnyMethod()
29             .AllowCredentials();
30     });
31 });
32 builder.Services.AddPooledDbContextFactory<GeoExplorerDbContext>(options =>
```

A2. COMUNICAÇÃO ENTRE FRONTEND E API NO MODO REAL

Fonte: src/frontend/src/services/apiGameDataSource.ts, linhas 49-73.

```
49 export function createApiGameDataSource(apiBaseUrl: string): GameDataSource {
50     return {
51         createSession(config: SessionConfig): Promise<CreateSessionResponse> {
52             return requestJson<CreateSessionResponse>(`${apiBaseUrl}/sessions`, {
53                 method: "POST",
54                 body: JSON.stringify(config),
55             });
56         },
57         getCurrentRound(sessionId: string): Promise<ChallengeRound> {
58             return requestJson<ChallengeRound>(`${apiBaseUrl}/sessions/${sessionId}/current-round`);
59         },
60         submitGuess(
61             sessionId: string,
```

```

64         roundId: string,
65         guess: GuessCoordinates
66     ): Promise<RoundResolutionResponse> {
67         return requestJson<RoundResolutionResponse>(
68             `${apiBaseUrl}/sessions/${sessionId}/rounds/${roundId}/guess`,
69             {
70                 method: "POST",
71                 body: JSON.stringify({ guess }),
72             }
73         );

```

A3. VALIDAÇÃO DE PALPITE NO BACKEND

Fonte: src/backend/Services/GameRoundRules.cs, linhas 216-236.

```

216     public static GuessCoordinatesDto ValidateGuess(GuessCoordinatesDto guess)
217     {
218         if (!double.IsFinite(guess.Latitude) || !double.IsFinite(guess.Longitude))
219         {
220             throw new GameFlowException("O palpite tem coordenadas inválidas.",
221 | StatusCodes.Status400BadRequest);
222         }
223         if (guess.Latitude is < MinGuessLatitude or > MaxGuessLatitude ||
224             guess.Longitude is < MinGuessLongitude or > MaxGuessLongitude)
225         {
226             throw new GameFlowException("O palpite tem coordenadas fora do mapa europeu.",
227 | StatusCodes.Status400BadRequest);
228         }
229         var label = string.IsNullOrWhiteSpace(guess.Label)
230             ? "Palpite"
231             : guess.Label.Trim();
232         if (label.Length > MaxGuessLabelLength || label.Any(char.IsControl))
233         {
234             throw new GameFlowException("A descrição do palpite não é válida.",
235 | StatusCodes.Status400BadRequest);
236         }

```

A4. CATÁLOGO POSTGRESQL ANTES DO SEED JSON

Fonte: src/backend/Services/SeedLocationCatalog.cs, linhas 70-99.

```

70         if (store is null)
71         {
72             logger.LogWarning("PostgreSQL catalog enabled, but no LocationCatalogStore was
configured.");
73             return LoadFromJson(environment);
74         }
75         if (File.Exists(GetSeedPath(environment)))
76         {
77             var fileLocations = LoadFromJson(environment);
78             try
79             {
80                 var importedLocations = store
81                     .ImportAndLoadAsync(fileLocations)
82                     .GetAwaiter()
83                     .GetResult();
84                 logger.LogInformation("Upserted and loaded {LocationCount} seed locations from
85 | PostgreSQL.", importedLocations.Count);
86                 return importedLocations;
87             }
88             catch (Exception exception) when (exception is not InvalidOperationException)
89             {
90                 logger.LogWarning(
91                     exception,
92                     "Could not upsert locations into PostgreSQL. Falling back to existing
PostgreSQL rows
93 | or locations.json.");
94                 var databaseLocations = TryLoadAllFromPostgres(logger, store);
95                 return databaseLocations.Count > 0 ? databaseLocations : fileLocations;
96             }
97         }
98     }
99

```

A5. SCHEMA EF DAS TABELAS PRINCIPAIS

Fonte: src/backend/Data/GeoExplorerDbContext.cs, linhas 12-18.

```
12     public DbSet<LocationEntity> Locations => Set<LocationEntity>();
13     public DbSet<GameSessionEntity> GameSessions => Set<GameSessionEntity>();
14     public DbSet<SessionRoundEntity> SessionRounds => Set<SessionRoundEntity>();
15     public DbSet<MultiplayerRoomEntity> MultiplayerRooms => Set<MultiplayerRoomEntity>();
16     public DbSet<MultiplayerPlayerEntity> MultiplayerPlayers => Set<MultiplayerPlayerEntity>();
17     public DbSet<MultiplayerRoundEntity> MultiplayerRounds => Set<MultiplayerRoundEntity>();
18     public DbSet<MultiplayerGuessEntity> MultiplayerGuesses => Set<MultiplayerGuessEntity>();
```

A6. EVENTOS EM TEMPO REAL DO HUB MULTIPLAYER

Fonte: src/backend/Hubs/MultiplayerHub.cs, linhas 84-97.

```
84     public async Task<MultiplayerRoomStateDto> SubmitGuess(SubmitMultiplayerGuessRequest request)
85     {
86         var update = await HandleGameFlow("SubmitGuess", request, () =>
_rooms.SubmitGuessAsync(request));
87         await BroadcastUpdate(update);
88         return update.State;
89     }
90
91     public async Task<MultiplayerRoomStateDto> ReadyForNextRound(MultiplayerRoomPlayerRequest
request)
92     {
93         var update = await HandleGameFlow("ReadyForNextRound", request, () =>
| _rooms.ReadyForNextRoundAsync(request));
94         await BroadcastUpdate(update);
95         await BroadcastOpenRooms();
96         return update.State;
97     }
```

A7. PERFIL FULL DO DOCKER COMPOSE

Fonte: docker-compose.yml, linhas 17-49.

```
17     frontend-api:
18       profiles: ["full"]
19       build:
20         context: .
21         dockerfile: src/frontend/Dockerfile
22         target: development
23       environment:
24         VITE_DATA_MODE: api
25         VITE_API_BASE_URL: /api
26         VITE_PROXY_TARGET: http://backend:8080
27       depends_on:
28         - backend
29       ports:
30         - "${FRONTEND_PORT:-5173}:5173"
31       volumes:
32         - ./src/frontend:/workspace/src/frontend
33         - frontend-node-modules:/workspace/src/frontend/node_modules
34
35     backend:
36       profiles: ["full"]
37       build:
38         context: .
39         dockerfile: src/backend/Dockerfile
40         target: development
41       environment:
42         ASPNETCORE_ENVIRONMENT: ${ASPNETCORE_ENVIRONMENT:-Development}
43         ASPNETCORE_HTTP_PORT: ${ASPNETCORE_HTTP_PORT:-8080}
44         GeoExplorer__UsePostgresCatalog: "true"
45         GeoExplorer__UsePostgresPersistence: "true"
46         GeoExplorer__ExposeDatabaseDiagnostics: ${GeoExplorer__ExposeDatabaseDiagnostics:-false}
47         GeoExplorer__AllowedOrigins:
| ${GeoExplorer__AllowedOrigins:-http://localhost:5173;http://127.0.0.1:5173}
48         ConnectionStrings__GeoExplorerDb: Host=db;Port=5432;Database=${POSTGRES_DB:-
| geoeexplorer};Username=${POSTGRES_USER:-geoeexplorer};Password=${POSTGRES_PASSWORD:-
geoeexplorer_dev}
49         MAPILLARY_ACCESS_TOKEN: ${MAPILLARY_ACCESS_TOKEN:-}
```

Anexo B - Comandos de validação

Durante a validação final, usei comandos deste tipo para confirmar a aplicação e o relatório:

```
dotnet test GeoExplorer.slnx
cd src/frontend && npm test -- --run
cd src/frontend && npm run build
node src/database/tools/audit-location-dataset.mjs --fail-on-errors
docker compose --profile full config
docker compose --profile full up --build
dotnet list GeoExplorer.slnx package --vulnerable
cd src/frontend && npm audit
```